SQL y T-SQL desde cero

Un texto para leer de corrido antes de ponerte a hacer ejercicios. Reemplaza las clases grabadas de la práctica (consultas, JOINs, subconsultas, operadores de conjunto, funciones, procedimientos y triggers).

Cómo usar este libro

Este texto sigue el mismo camino que siguieron las clases: empieza por la consulta más simple posible y en cada capítulo agrega **una sola idea nueva**, que se apoya en las anteriores. Por eso conviene leerlo en orden la primera vez. Al final de cada capítulo hay un recuadro "Para llevarte" con lo mínimo que tenés que recordar.

El recorrido es este:

- **Capítulos 1 a 4. Preguntarle cosas a una tabla.** Primero conocemos la base con la que trabaja la materia. Después aprendemos a pedir columnas y filas, a transformar datos con funciones y a resumir con agrupaciones.
- **Capítulo 5. Cambiar datos.** INSERT, UPDATE y DELETE, y la primera idea de transacción.
- **Capítulos 6 y 7. Combinar tablas.** Los JOINs, las subconsultas y los operadores de conjunto (UNION, INTERSECT, EXCEPT). Acá aparece el problema más importante de toda la práctica: la **multiplicación de filas**.
- **Capítulo 8. Resolver consultas de parcial.** Un método de cinco pasos y un recetario de patrones, cada uno con un ejercicio resuelto y probado contra tu base.
- **Capítulos 9 a 12. Programar dentro de la base.** Funciones, cursores, procedimientos, transacciones, triggers, y cómo elegir el objeto que pide cada enunciado.
- **Capítulo 13. Plan de práctica.**
- **Apéndice.** Los tres parciales anteriores resueltos. Leelo **después** de intentarlos.

Convenciones:

- 🎙️ marca lo que dijo el profe en clase: sus criterios, sus advertencias y sus recomendaciones. Es lo que más probablemente te van a evaluar.
- ⚠️ marca una trampa frecuente.
- Todo el código corre en SQL Server contra tu base `GESTION2016`. Cuando aparece un resultado ("devuelve 31 filas"), es lo que da en tu base.
- 🎙️ La base del profe tiene algunos datos más que la tuya (por ejemplo, facturas de 2025). Si un resultado tuyo difiere en una o dos filas del de él, es normal.

Cuando termines de leer y quieras algo corto para tener abierto durante el parcial, usá el otro archivo, Parcial-2-ResumenParaElParcial.md.

Capítulo 1 — La base de la materia

Antes de escribir una línea de SQL tenés que tener la base en la cabeza como si fuera un lugar físico. 🎙️ El profe lo dijo en la clase de JOINs: *"no solamente hay que saber SQL, también hay que conocer el modelo"*. La guía, el TP y los parciales trabajan sobre esta base.

1.1 Un supermercado

`GESTION2016` es el sistema de gestión de una distribuidora: vende productos a clientes, los guarda en depósitos y tiene empleados. Pensalo como un supermercado:

Tabla	Metáfora	Una fila es…
`Factura`	el **ticket** (cabecera)	una compra: fecha, cliente, vendedor, total
`Item_Factura`	los **renglones del ticket**	un producto dentro de una compra: cantidad y precio unitario
`Producto`	la **góndola**	un producto (con familia, rubro y envase)
`Composicion`	la **receta del combo**	"el combo X lleva N unidades del componente Y"
`STOCK`	el **inventario por depósito**	cuánto hay del producto P en el depósito D
`DEPOSITO`	los **depósitos**	uno, con su encargado (un empleado) y su zona
`Empleado`	el **organigrama**	un empleado con su jefe (`empl_jefe` apunta a otro empleado)
`Cliente`	la **cartera de clientes**	un cliente con su vendedor asignado y su límite de crédito
`Familia`, `Rubro`, `Zona`	**clasificaciones**	una categoría con su descripción

Las columnas llevan un prefijo con la tabla: `clie_codigo`, `fact_fecha`, `prod_detalle`, `stoc_cantidad`. Así, al leer una columna sabés de qué tabla viene.

1.2 Tres tipos de tablas

🎙️ En la clase de subconsultas el profe clasificó las tablas en tres tipos. Tenerlo presente te ayuda a prever cuánto pesa una consulta:

- **Transaccionales:** se llenan con *operaciones* (leer un código de barras, pagar con billetera virtual, sacar plata de un cajero). Son las que crecen sin parar y tienen el volumen. En tu base son `Factura` e `Item_Factura`.
- **Maestras:** los datos propios del negocio, que cambian poco: `Cliente`, `Producto`, `STOCK`, `DEPOSITO`, `Empleado`.
- **Paramétricas:** clasificaciones y parámetros: `Familia`, `Rubro`, `Zona`.

1.3 La clave del ticket tiene tres partes

Una factura no se identifica solo por su número: hay facturas A y B, de distintas sucursales, y el mismo número puede repetirse. La clave primaria de `Factura` es `(fact_tipo, fact_sucursal, fact_numero)`, y cada renglón la repite en `(item_tipo, item_sucursal, item_numero)`. Esto vuelve a aparecer en el capítulo 6, y es la causa de muchos errores.

1.4 La tabla Composicion

🎙️ El profe la explicó con una hamburguesería. Un combo lleva hamburguesa, papas y gaseosa, y la hamburguesa a su vez lleva carne, lechuga y tomate. *"Nunca te van a vender una rodaja de tomate"*, pero el tomate tiene proveedores y stock, y si falta no podés vender la hamburguesa. En tu base hay casos así: "Linterna con pilas" se compone de 1 linterna y 2 pilas.

Cada fila de `Composicion` dice: el producto `comp_producto` lleva `comp_cantidad` unidades del producto `comp_componente`. Las dos columnas apuntan a `Producto`. Un componente también puede ser compuesto (composición en cascada), y por eso más adelante aparecen las funciones recursivas.

1.5 Los datos de tu base

Las facturas van de 2010 a 2012, y 2010 tiene muy pocas. Si un enunciado dice "2025" o "el año actual", probá con 2012, y en el parcial escribí el año que pide (o `YEAR(GETDATE())`).

> **Para llevarte**
>
> - Ticket = `Factura`; renglones = `Item_Factura`; góndola = `Producto`; inventario = `STOCK`.
> - La factura se identifica con **tres** columnas.
> - Las transaccionales (`Factura`, `Item_Factura`) son las pesadas.

Capítulo 2 — Preguntarle cosas a una tabla

2.1 SELECT y FROM

La consulta mínima tiene dos partes obligatorias: **qué** querés ver (`SELECT`) y **de dónde** (`FROM`).

```
SELECT * FROM Cliente
```

El asterisco trae todas las columnas (Cliente tiene 6) y, como no hay ningún filtro, todas las filas (unas 3.700). Para ver solo algunas columnas, se nombran:

```
SELECT clie_codigo, clie_razon_social, clie_telefono, clie_limite_credito
FROM Cliente
```

Elegir columnas no filtra filas: seguís viendo todas las filas, con menos columnas.

🎙️ **Regla de fondo:** un SELECT **nunca modifica la tabla**. Todo lo que vas a ver en este capítulo (sacar duplicados, cambiar nombres, resumir) pasa en el *resultado* de la consulta. Los datos guardados no se tocan.

2.2 DISTINCT: sacar duplicados

```
SELECT DISTINCT clie_vendedor FROM Cliente
```

Devuelve cada vendedor una sola vez. Dos detalles que marcó el profe:

- **NULL cuenta como un valor más:** si hay muchos clientes sin vendedor, el NULL aparece **una** vez.
- Con varias columnas (`SELECT DISTINCT clie_vendedor, clie_codigo`), lo que no se repite es la **combinación**. Cada vendedor vuelve a aparecer muchas veces, una por cliente.

2.3 Alias de columna

Un alias cambia el nombre de la columna en el resultado:

```
SELECT clie_razon_social AS [Razón social], clie_telefono AS telefono, clie_codigo
FROM Cliente
```

- Un nombre de columna de tabla no puede tener espacios. Un alias sí (entre corchetes).
- Ponerle alias a una columna no te obliga a ponérselo a las demás.
- El `AS` es opcional (`clie_telefono telefono` funciona igual).

🎙️ *"Hay que ser prudente con el uso de los alias"*. Si a `ISNULL(precio, 0)` le ponés el alias `precio` , más adelante no queda claro si `precio` es la columna original o tu cálculo. No da error, pero es ambiguo.

2.4 CASE: mostrar algo que no está en la tabla

Hasta acá mostramos los datos tal como están guardados. `CASE` permite mostrar un valor calculado según condiciones:

```
SELECT clie_razon_social, clie_limite_credito,
    CASE WHEN clie_limite_credito < 100              THEN 'Poco'
         WHEN clie_limite_credito BETWEEN 100 AND 4000  THEN 'Moderado'
         ELSE 'Alto' END AS credito
FROM Cliente
```

Reglas del CASE:

1. Las condiciones se evalúan **en el orden en que están escritas**. La primera verdadera gana y las siguientes no se miran.
2. Si ninguna es verdadera y no hay `ELSE`, el resultado es **NULL**.
3. Todas las ramas tienen que devolver **el mismo tipo de dato**: si una devuelve texto, no puede haber otra que devuelva una fecha.
4. Sin alias, la columna aparece como "(Sin nombre de columna)". No es un error, pero queda feo.

2.5 WHERE: elegir filas

El `WHERE` filtra: solo quedan las filas para las que la condición da verdadero. Los operadores son estos:

Tipo	Positivo	Negado
comparación	`=`, `<`, `>`, `<=`, `>=`	`<>` (y `>=` es la negación de `<`, etc.)
rango	`BETWEEN a AND b`	`NOT BETWEEN`
nulidad	`IS NULL`	`IS NOT NULL`
lista	`IN (v1, v2, ...)`	`NOT IN`
texto	`LIKE 'patrón'`	`NOT LIKE`

- `BETWEEN` **incluye los extremos**: `BETWEEN 100 AND 4000` es verdadero para 100 y para 4000.
- 🎙️ Todos sirven para cualquier tipo de dato. Podés pedir un rango de fechas (`BETWEEN '2012-04-01' AND '2012-04-30'`), de números o de textos (`BETWEEN 'García' AND 'González'`).
- `LIKE` usa dos comodines: `%` es *cualquier cantidad de caracteres* y `_` es *exactamente uno*.
    - `LIKE 'Marcel%'` encuentra Marcel, Marcelo y Marcela (empieza con).
    - `LIKE 'Marcel_'` encuentra Marcelo y Marcela, pero no Marcel (exactamente un carácter más).
    - `LIKE '%mar%'` encuentra todo lo que contenga "mar", incluidas Tamara y María (contiene).

Las condiciones se combinan con `AND` (tienen que cumplirse todas) y `OR` (alcanza con una). ⚠️ **AND tiene prioridad sobre OR.** Esta consulta:

```
SELECT * FROM Cliente
WHERE clie_vendedor = 3 OR clie_vendedor = 7 AND clie_razon_social LIKE '%MARISOL%'
```

el motor la lee como `vendedor = 3 OR (vendedor = 7 AND ...)`. Si lo que querías era "del vendedor 3 o 7, y que se llame Marisol", hacen falta paréntesis:

```
SELECT * FROM Cliente
WHERE (clie_vendedor = 3 OR clie_vendedor = 7) AND clie_razon_social LIKE '%MARISOL%'
```

🎙️ *"Si los combinamos, es conveniente ponerlo entre paréntesis."*

2.6 NULL: la ausencia de valor

NULL no es cero ni texto vacío: es **"no hay dato"**. Y tiene una regla que hay que aprender de memoria:

```
SELECT * FROM Cliente WHERE clie_vendedor = NULL    -- 0 filas, siempre
SELECT * FROM Cliente WHERE clie_vendedor IS NULL   -- los clientes sin vendedor
```

🎙️ La explicación del profe: los operadores de comparación *"están programados para trabajar únicamente con valores"*. Si de un lado hay un NULL, la comparación no da verdadera. Pasa con `=`, con `<>`, con `>` y hasta con `NULL = NULL`. Para preguntar por la ausencia de valor existen `IS NULL` e `IS NOT NULL`.

Vas a ver esta regla aparecer en lugares inesperados: en los JOINs (capítulo 6), en `NOT IN` y en comparaciones contra subconsultas (capítulo 7).

2.7 ORDER BY: el orden del resultado

Sin `ORDER BY` el motor devuelve las filas *"como las encontró primero en disco"*. Ese orden no está garantizado: cambia con las inserciones y con el uso. Si te importa el orden, lo pedís:

```
SELECT clie_vendedor, clie_limite_credito, clie_codigo
FROM Cliente
ORDER BY clie_vendedor, clie_limite_credito DESC, clie_codigo
```

- Se ordena por la primera columna, y **solo ante un empate** se mira la segunda (y así sucesivamente).
- Cada columna puede ser `ASC` (el default, no hace falta escribirlo) o `DESC`.
- **NULL** queda **primero** en orden ascendente y **último** en descendente.
- Se puede ordenar por una columna, por una función, por el **número de columna** del SELECT (`ORDER BY 2`) o por un alias.

🎙️ Criterios del profe:

- `ORDER BY 2` se refiere a la segunda columna **del SELECT**, no de la tabla. Si el SELECT tiene 2 columnas y ponés `ORDER BY 3`, da error antes de ejecutar. Y no podés usar el número si la columna por la que querés ordenar no se muestra.
- Ordenar por alias funciona, pero *"a mi gusto no es recomendable"*, por la ambigüedad del 2.3. Él prefiere el número de columna.

Para llevarte

- SELECT y FROM son obligatorios. Un SELECT no modifica nada.
- `= NULL` nunca es verdadero: se usa `IS NULL`.
- Mezclando AND y OR, poné paréntesis.
- Sin ORDER BY no hay orden garantizado.

Capítulo 3 — Funciones del motor

Hasta acá todo era SQL estándar (ANSI), que corre igual en cualquier motor. Desde este capítulo usamos **T-SQL**, el dialecto de SQL Server, que agrega funciones propias. Todavía no es programar: son funciones que se usan dentro de una consulta.

3.1 ISNULL

```
SELECT clie_codigo, clie_vendedor, ISNULL(clie_vendedor, 0) AS vendedor_o_cero
FROM Cliente
```

`ISNULL(x, reemplazo)` devuelve `x` si tiene valor y `reemplazo` si es NULL. El reemplazo tiene que ser del mismo tipo que `x`. Es la única función que nunca devuelve NULL (salvo que hagas la redundancia `ISNULL(x, NULL)`). Todas las demás, si reciben NULL, devuelven NULL: `YEAR(NULL)` es NULL, `UPPER(NULL)` es NULL.

3.2 Números, fechas y textos

Tipo	Función	Qué hace
números	`+ - * / %`	cuentas; ⚠ entero / entero da entero: `7 / 2 = 3`, `7 * 1.0 / 2 = 3.5`
	`ROUND(x, 2)`	redondea a 2 decimales
	`ABS(x)`	valor absoluto
fechas	`YEAR(f)`, `MONTH(f)`, `DAY(f)`	partes de una fecha
	`DATEDIFF(YEAR, f1, f2)`	diferencia entre fechas (en `DAY`, `MONTH`, `YEAR`…)
	`DATEADD(DAY, 45, f)`	suma (o resta) tiempo a una fecha
	`GETDATE()`	fecha y hora actual
textos	`UPPER`, `LOWER`	mayúsculas y minúsculas
	`RTRIM`, `LTRIM`	sacan espacios a derecha e izquierda
	`LEN(s)`	longitud (`LEN('')` = 0, `LEN(NULL)` = NULL)
	`SUBSTRING(s, desde, cuántos)`	el tercer argumento es una **longitud**, no la posición final
	`CHARINDEX(buscada, s)`	posición donde empieza (la primera es **1**); 0 si no está
	`CONCAT(a, b, …)` o `a + b`	concatena
	`REPLICATE('0', n)`	repite un texto n veces (sirve para rellenar con ceros)
conversión	`CAST(x AS INT)`	convierte de tipo; puede fallar **al ejecutar** si el dato no se puede convertir
	`CONVERT(VARCHAR, f, código)`	da formato a una fecha según un código de estilo

Un ejemplo de la clase: muchos campos de texto son `CHAR`, de longitud fija, y vienen con espacios a la derecha ("González "). Por eso aparece tanto `RTRIM`:

```
SELECT CONCAT(UPPER(RTRIM(empl_apellido)), ', ', RTRIM(empl_nombre)) AS empleado,
    DATEDIFF(YEAR, empl_nacimiento, empl_ingreso)                    AS edad_al_ingresar
FROM Empleado
ORDER BY 2
```

3.3 Dónde se puede usar una función

🎙️ Una función devuelve un valor, así que puede ir en cualquier lugar donde iría un valor: **SELECT, WHERE, GROUP BY, HAVING y ORDER BY**. No va en el FROM, porque ahí van tablas. El profe lo resumió en dos frases: *"todo lo que se puede poner en el WHERE se puede poner en el HAVING"* y *"todo lo que se puede poner en el SELECT se puede poner en el ORDER BY"*.

> **Para llevarte**
>
> - `ISNULL(x, 0)` para reemplazar nulos. El resto de las funciones propagan el NULL.
> - Ojo con la división entera: multiplicá por `1.0` o `100.0`.
> - `CHAR` trae espacios: `RTRIM`.

Capítulo 4 — Resumir: agregación, GROUP BY y HAVING

Hasta ahora cada fila del resultado era una fila de la tabla. Ahora vamos a pedir **resúmenes**: cuántos, cuánto suman, cuál es el máximo.

4.1 Funciones de agregación

🎙️ La idea del profe fue esta: dividís las filas en **conjuntos** y a cada conjunto le aplicás una función que devuelve un único valor. Su ejemplo fue agrupar clientes por localidad (Lomas de Zamora, Quilmes, Capital, Vicente López y los que no tienen domicilio) y preguntar "¿cuántos viven en cada una?".

Función	Qué devuelve	Tipos
`COUNT(*)`	cantidad de **filas**	—
`COUNT(col)`	cantidad de filas donde `col` **no es NULL**	cualquiera
`COUNT(DISTINCT col)`	cantidad de valores distintos no nulos	cualquiera
`MIN(col)` , `MAX(col)`	el menor y el mayor	cualquiera (números, fechas, textos)
`SUM(col)` , `AVG(col)`	suma y promedio	solo números

`AVG` y `COUNT(col)` **ignoran los NULL**: el promedio se calcula sobre los que tienen valor, y los NULL no cuentan como cero.

Sin GROUP BY, toda la tabla (o lo que quede después del WHERE) es **un solo conjunto**, y el resultado es **una fila**:

```
SELECT COUNT(*)                 AS clientes,
    COUNT(clie_vendedor)        AS con_vendedor,
    COUNT(DISTINCT clie_vendedor) AS vendedores,
    MIN(clie_limite_credito), MAX(clie_limite_credito), AVG(clie_limite_credito)
FROM Cliente
```

4.2 GROUP BY: un conjunto por valor

```
SELECT clie_vendedor, COUNT(*) AS clientes
FROM Cliente
GROUP BY clie_vendedor
```

Ahora hay **un grupo por cada vendedor** (más un grupo para los NULL) y el resultado tiene **una fila por grupo**. Si agregás un WHERE, primero se filtran las filas y después se arman los grupos con lo que quedó.

⚠️ El error más clásico es este:

```
SELECT clie_vendedor, clie_razon_social, COUNT(*)   -- ERROR
FROM Cliente
GROUP BY clie_vendedor
```

🎙️ El profe lo explicó así: pediste *una fila por vendedor*, pero un vendedor tiene 20 clientes, es decir, 20 razones sociales. ¿Cuál mostrarías? *"No tiene lógica poner este campo acá."* Una vez que agrupaste, *"ya no dispongo de la granularidad"*: después del GROUP BY solo podés usar **las columnas por las que agrupaste** y **funciones de agregación**.

🎙️ Por eso, cuando querés mostrar código y descripción, agrupás por las dos. Agregar al GROUP BY una columna que depende de la clave (como el detalle de un producto) **no cambia la cantidad de grupos** y te deja mostrarla:

```
SELECT p.prod_codigo, p.prod_detalle, COUNT(*) AS componentes
FROM Producto p JOIN Composicion c ON c.comp_producto = p.prod_codigo
GROUP BY p.prod_codigo, p.prod_detalle
```

(El JOIN se ve en el capítulo 6. Fijate solamente en el GROUP BY.)

🎙️ Para sacar repetidos, el profe prefiere **agrupar** antes que `DISTINCT` (*"el DISTINCT no es lo recomendable"*), porque con el GROUP BY después podés agregar sumas y conteos.

4.3 HAVING: filtrar grupos

`WHERE` filtra **filas** y se ejecuta antes de agrupar. `HAVING` filtra **grupos** y se ejecuta después, así que puede usar funciones de agregación:

```
SELECT MONTH(fact_fecha) AS mes, COUNT(*) AS facturas
FROM Factura
WHERE YEAR(fact_fecha) = 2011        -- filas: solo 2011
GROUP BY MONTH(fact_fecha)           -- un grupo por mes
HAVING COUNT(*) > 15                 -- grupos: meses con más de 15 facturas
ORDER BY 1
```

- ⚠️ Un agregado en el WHERE da error: en ese momento todavía no hay grupos.
- 🎙️ *"Una cosa es lo que muestro y otra cosa es lo que filtro."* Podés filtrar por columnas o agregados que no aparecen en el SELECT.

4.4 El orden en que el motor resuelve una consulta

Ahora sí podemos ver la consulta completa. Se **escribe** en este orden:

```
SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
```

pero el motor la **resuelve** en este otro:

1. **FROM**: de qué tablas salen las filas.
2. **WHERE**: qué filas quedan.
3. **GROUP BY**: cómo se agrupan.
4. **HAVING**: qué grupos quedan.
5. **SELECT**: qué se calcula y se muestra.
6. **ORDER BY**: en qué orden.

🎙️ La lógica es la que explicó el profe: no podés agrupar sin tener las filas, no podés filtrar grupos sin haber agrupado, y *"no puedo ordenar hasta que no sepa lo que tengo que mostrar"*. Este orden explica muchas cosas: por qué el WHERE no ve los agregados, por qué el ORDER BY sí ve

los alias del SELECT, y (en el capítulo 7) cuándo se ejecuta cada subconsulta.

4.5 Agregados sobre un conjunto vacío

Este detalle parece menor, pero en el capítulo 7 vuelve como trampa. 🎙️ El profe lo mostró con una condición imposible (`WHERE 1 = 2`):

```
SELECT COUNT(*), MIN(clie_limite_credito) FROM Cliente WHERE 1 = 2
-- 1 fila: 0, NULL

SELECT clie_vendedor, COUNT(*) FROM Cliente WHERE 1 = 2 GROUP BY clie_vendedor
-- 0 filas
```

- **Sin GROUP BY**, la función se aplica sobre el conjunto (aunque esté vacío) y **siempre devuelve una fila**: `COUNT` da 0, y `SUM` , `MIN` , `MAX` y `AVG` dan NULL.
- **Con GROUP BY**, si no quedan filas no se puede armar ningún grupo, así que la función nunca se ejecuta y el resultado tiene **0 filas**.

> **Para llevarte**
>
> - Después del GROUP BY solo quedan las columnas agrupadas y los agregados.
> - WHERE filtra filas (antes) y HAVING filtra grupos (después).
> - Orden real: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.
> - Un agregado sin GROUP BY devuelve **siempre** una fila; con GROUP BY puede devolver 0.

Capítulo 5 — Cambiar datos: INSERT, UPDATE, DELETE

Hasta acá solo leímos. Estas tres instrucciones modifican la tabla, y por eso el motor las trata con un cuidado especial: las **transacciones**.

5.1 Las tres instrucciones

```
-- agregar filas: una...
INSERT INTO Cliente (clie_codigo, clie_razon_social, clie_limite_credito)
VALUES ('99999', 'PRUEBA SA', 1000)

-- ...o muchas, a partir de un SELECT (misma cantidad de columnas, tipos compatibles)
INSERT INTO TablaDestino (col1, col2)
SELECT algo, otra_cosa FROM ... WHERE ...

-- modificar
UPDATE Cliente
SET clie_limite_credito = clie_limite_credito * 1.1,
   clie_telefono = NULL
WHERE clie_vendedor = 3

-- borrar
DELETE FROM Cliente WHERE clie_codigo = '99999'
```

Detalles que se preguntan:

- En el INSERT, si no ponés la lista de columnas, se asume **todas, en el orden en que se creó la tabla**. Si la ponés, las columnas que dejás afuera toman NULL o su valor por defecto.
- Una columna **IDENTITY** (autonumérica) no se pone en el INSERT: la completa el motor.
- Una columna con **DEFAULT** toma ese valor solo si la omitís. Si la nombrás con `NULL` explícito, queda NULL. 🎙️ El DEFAULT *no es una restricción*: no impide nada, solo completa.
- `UPDATE` y `DELETE` **sin WHERE** afectan a toda la tabla.
- Si el WHERE no encuentra filas, no hay error: "0 filas afectadas". Esto vuelve en el capítulo 10.
- En el `SET` de un UPDATE puede ir cualquier expresión: constantes, funciones, cálculos con la propia columna y, como vas a ver en el capítulo 7, subconsultas.

5.2 Transacciones: todo o nada

Una **transacción** es un conjunto de operaciones que se ejecutan de forma **atómica**: o se hacen todas, o no se hace ninguna.

Si no indicás nada, SQL Server trabaja en modo **autocommit**: **cada instrucción es una transacción**. Un `INSERT ... SELECT` que inserta 500 filas inserta las 500 o ninguna. 🎙️ El profe lo mostró con un INSERT de dos filas, una válida y otra con un texto demasiado largo para la columna: **la válida tampoco quedó**. El motor la había insertado y la deshizo.

Un DELETE también puede fallar entero. Si querés borrar un cliente que tiene facturas, la clave foránea lo impide, porque dejaría facturas *"huérfanas"*, y no se borra ninguna fila.

En el capítulo 10 vas a aprender a marcar vos dónde empieza y dónde termina una transacción, para agrupar varias instrucciones.

Para llevarte

- Sin WHERE, UPDATE y DELETE tocan toda la tabla.
- Cada instrucción es atómica: si falla una fila, no queda ninguna.

Capítulo 6 — Combinar tablas: JOINs

Casi ninguna pregunta interesante se responde con una sola tabla. "¿Qué productos se vendieron en 2012?" necesita el producto (en `Producto`), la venta (en `Item_Factura`) y la fecha (en `Factura`). Para juntarlas está el JOIN.

6.1 El punto de partida: el producto cartesiano

Si ponés dos tablas en el FROM sin decir cómo se relacionan, el motor combina **todas las filas de una con todas las de la otra**. 🎙️ El ejemplo del profe: una tabla con los 7 días de la semana y otra con 3 turnos (0-8, 8-16, 16-24) dan 21 filas. **Las columnas se suman y las filas se multiplican.**

```
SELECT * FROM Cliente, Factura    -- cada cliente con TODAS las facturas
```

Esto *compila y corre*, pero 🎙️ *"semánticamente no significa absolutamente nada"*: le pega a cada cliente facturas que no son suyas. Un JOIN es un producto cartesiano más una condición que deja solo las combinaciones con sentido.

6.2 INNER JOIN

```
SELECT f.fact_numero, f.fact_fecha, c.clie_razon_social
FROM Factura f
JOIN Cliente c ON c.clie_codigo = f.fact_cliente
```

`JOIN` es lo mismo que `INNER JOIN`. En el ON va la condición que relaciona las tablas. 🎙️ Normalmente es **clave foránea = clave primaria**: `fact_cliente` (en Factura) apunta a `clie_codigo` (en Cliente).

Solo quedan las combinaciones que cumplen el ON. Por eso:

- Un cliente **sin facturas** no aparece.
- Una factura **con `fact_cliente` en NULL** tampoco aparece: por la regla del capítulo 2, `NULL = algo` no es verdadero.

Alias de tabla. `Factura f` le pone a la tabla el apodo `f`. Hacen falta cuando dos tablas tienen columnas con el mismo nombre, y son cómodos siempre. 🎙️ Una vez que ponés un alias, estás obligado a usarlo: el nombre original de la tabla deja de valer en esa consulta.

Más de dos tablas. Los joins se encadenan y se resuelven de izquierda a derecha. Estos son los tres que vas a escribir el 90% de las veces:

```
-- renglón → ticket (clave de 3 partes, SIEMPRE las tres)
JOIN Factura f ON f.fact_tipo = i.item_tipo AND f.fact_sucursal = i.item_sucursal AND f.fact_numero = i.item_numero
-- renglón → producto
JOIN Producto p ON p.prod_codigo = i.item_producto
-- producto → stock
JOIN STOCK s ON s.stoc_producto = p.prod_codigo
```

⚠️ Si unís renglones con facturas solo por el número, cada renglón se cruza con *todas* las facturas que tienen ese número en otros tipos y sucursales: un producto cartesiano parcial. Siempre van las **tres** columnas.

Ejemplo completo (ejercicio 2 de la guía, resuelto en clase): código, detalle y cantidad vendida de los productos vendidos en 2012.

```
SELECT p.prod_codigo, p.prod_detalle, SUM(i.item_cantidad) AS cantidad_vendida
FROM Producto p
JOIN Item_Factura i ON i.item_producto = p.prod_codigo
JOIN Factura f ON f.fact_tipo = i.item_tipo AND f.fact_sucursal = i.item_sucursal AND f.fact_numero = i.item_numero
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY p.prod_codigo, p.prod_detalle
ORDER BY 3
```

La fecha no está en el renglón sino en el ticket. Por eso, aunque la pregunta es sobre productos, hay que pasar por `Factura`. Y como un producto se vendió muchas veces, el join lo repite: 🎙️ el profe agrupó (en vez de usar DISTINCT) para tener una fila por producto y poder sumar.

6.3 El problema central: la multiplicación de filas

Esta sección es la más importante del capítulo.

Cuando unís una tabla con otra que tiene **varias filas por cada una** (un cliente con sus facturas, una factura con sus renglones), las filas de la primera **se repiten**. Eso a veces es justo lo que querés y a veces arruina el resultado.

🎙️ El ejemplo del profe:

- Cliente JOIN Factura agrupado por cliente: `COUNT(*)` da la cantidad de facturas. Bien.
- Si además unís Item_Factura, cada factura se repite una vez por renglón. Ahora `COUNT(*)` cuenta **renglones**, no facturas, y `SUM(fact_total)` suma el total de cada factura **tantas veces como renglones tiene**.

Peor todavía: si unís un producto con `STOCK` (digamos 10 depósitos) **y** con `Item_Factura` (200 ventas), cada producto aparece 10 × 200 = 2.000 veces. Cualquier suma de ventas sale multiplicada por 10, y cualquier suma de stock, por 200.

🎙️ **Qué se rompe y qué no:** *"lo que se rompe con la multiplicación son las sumatorias, los promedios y los count; el mínimo y el máximo no"*. Repetir un valor no cambia cuál es el mayor. Por eso, en el ejercicio 7 de la guía (precio máximo y mínimo de cada producto con stock), el profe unió Producto con STOCK **y** con Item_Factura sin problema:

```
SELECT p.prod_codigo, p.prod_detalle,
    MAX(i.item_precio) AS maximo, MIN(i.item_precio) AS minimo,
    (MAX(i.item_precio) - MIN(i.item_precio)) * 100 / MIN(i.item_precio) AS porc_diferencia
FROM Producto p
JOIN STOCK s ON s.stoc_producto = p.prod_codigo
JOIN Item_Factura i ON i.item_producto = p.prod_codigo
WHERE s.stoc_cantidad > 0          -- "que posean stock": la fila puede existir con 0
GROUP BY p.prod_codigo, p.prod_detalle
```

Si te pidieran **sumas** de dos tablas hijas distintas, la solución es poner una en el FROM y calcular la otra con una subconsulta (capítulo 7). Esta regla es la base del método del capítulo 8.

6.4 OUTER JOIN: que no se pierda nadie

A veces querés *todos* los clientes, compren o no. Para eso está el `LEFT JOIN` (o `LEFT OUTER JOIN`):

```
SELECT c.clie_codigo, c.clie_razon_social, f.fact_numero, f.fact_total
FROM Cliente c
LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
```

🎙️ El vocabulario del profe es tabla **dominante** y tabla **dominada**. La dominante (en un LEFT, la de la izquierda) conserva **al menos una fila por cada registro suyo**. Si un cliente no tiene facturas, aparece una vez con todas las columnas de Factura en NULL. `RIGHT JOIN` es lo mismo con la dominante a la derecha. 🎙️ El profe recomienda usar siempre LEFT con la dominante en el FROM, por prolijidad. `FULL JOIN` hace que las dos sean dominantes.

En la dominada **todas** las columnas pueden venir en NULL, incluidas su clave primaria y sus columnas obligatorias. Por eso, para sumar, se usa `ISNULL`:

```
SELECT c.clie_codigo, c.clie_razon_social, SUM(ISNULL(f.fact_total, 0)) AS total
FROM Cliente c
LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
GROUP BY c.clie_codigo, c.clie_razon_social
```

⚠️ **La trampa del LEFT JOIN.** Querés todos los clientes con lo que compraron *en 2012*, y escribís:

```
FROM Cliente c LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
```

El WHERE se aplica **después** del join. Para los clientes sin facturas, `f.fact_fecha` es NULL, la condición no da verdadera y la fila se elimina. 🎙️ El LEFT *"terminó en un sinsentido"*: funciona como un INNER. La solución del profe es **achicar la dominada en el ON**:

```
FROM Cliente c LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
                                  AND YEAR(f.fact_fecha) = 2012
```

Así, primero se queda solo con las facturas de 2012 y recién después hace el LEFT. Un filtro sobre la **dominante** (por ejemplo, "que no sean consumidores finales") sí puede ir en el WHERE: 🎙️ *"la tabla dominante no necesariamente tiene que ir completa"*.

Otro ejemplo de la guía (ejercicio 3: stock total de cada producto). Si un producto no tiene ninguna fila en STOCK, con INNER desaparecería. 🎙️ *"Para ser estricto con el enunciado"*, el profe usó LEFT:

```
SELECT p.prod_codigo, p.prod_detalle, SUM(ISNULL(s.stoc_cantidad, 0)) AS stock_total
FROM Producto p
LEFT JOIN STOCK s ON s.stoc_producto = p.prod_codigo
GROUP BY p.prod_codigo, p.prod_detalle
ORDER BY 2
```

6.5 SELF-JOIN: la tabla consigo misma

A veces los dos lados de la relación están en la **misma** tabla. El caso típico es el empleado y su jefe: `empl_jefe` es una clave foránea que apunta a la propia tabla `Empleado`. La tabla se nombra dos veces, y el alias es **obligatorio** (🧵 *"más prolijo ponerlo en ambas"*):

```
SELECT e.empl_codigo, RTRIM(e.empl_apellido) + ', ' + RTRIM(e.empl_nombre) AS empleado,
    RTRIM(j.empl_apellido) + ', ' + RTRIM(j.empl_nombre) AS jefe
FROM Empleado e
LEFT JOIN Empleado j ON j.empl_codigo = e.empl_jefe
```

El truco mental: no es "la tabla con ella misma", son **dos roles** que casualmente viven en la misma tabla. Imaginá dos fotocopias, una con el cartel "empleado" y otra con el cartel "jefe". `Composicion` funciona igual, con `Producto` dos veces: una como *combo* (por `comp_producto`) y otra como *componente* (por `comp_componente`). Así lo resolvió el profe en el ejercicio 13:

```
SELECT combo.prod_detalle, combo.prod_precio,
    SUM(c.comp_cantidad * comp.prod_precio) AS precio_por_componentes
FROM Composicion c
JOIN Producto combo ON combo.prod_codigo = c.comp_producto
JOIN Producto comp  ON comp.prod_codigo  = c.comp_componente
GROUP BY combo.prod_codigo, combo.prod_detalle, combo.prod_precio
HAVING SUM(c.comp_cantidad) > 2        -- linterna (1) + pilas (2) = 3 > 2
ORDER BY SUM(c.comp_cantidad) DESC
```

⚠️ En clase, el primer intento tenía los alias cruzados (el segundo join usaba el alias del primero). Con la misma tabla dos veces, revisá que cada ON use su propio alias.

El tercer uso es buscar **pares** de renglones del mismo ticket ("productos que se venden juntos"), que se desarrolla en el capítulo 8.

Para llevarte

- JOIN = producto cartesiano + condición. Igualá FK con PK y la factura por sus **tres** columnas.
- Unir con una tabla "de muchos" repite filas: SUM, AVG y COUNT se inflan; MIN y MAX no.
- LEFT JOIN: el filtro de la dominada va en el **ON**, no en el WHERE.
- Self-join = dos roles, dos alias.

Capítulo 7 — Subconsultas, vistas y operadores de conjunto

Una **subconsulta** (o *subselect*) es un SELECT dentro de otro, entre paréntesis. 🎙️ *"Y acá terminado el misterio de la sintaxis."* Lo difícil no es escribirla: es saber **qué tiene que devolver** según dónde la pongas y **cuánto cuesta**.

7.1 Disco y memoria

🎙️ El profe arrancó la clase con esta idea: las tablas se leen de **disco**, y ahí hay índices que permiten encontrar rápido lo que buscás. Los resultados intermedios y el ordenamiento se manejan en **memoria**, que es más rápida pero **no tiene índices**. Lo que devuelve una subconsulta va a memoria. Por eso: *"toda la clase de hoy entera es cuidado con la memoria"*. Si una subconsulta sobre una tabla transaccional devuelve muchísimas filas, todo eso queda en memoria sin índices que ayuden.

7.2 Qué tiene que devolver, según dónde va

Dónde la ponés	Qué tiene que devolver
`WHERE x IN (subconsulta)` / `NOT IN`	**una columna**, cualquier cantidad de filas (incluso 0)
`WHERE x = (subconsulta)` , `>` , `<`	**un solo valor** (si puede traer varias filas, da error: usá IN)
`SELECT (subconsulta) AS col`	**una fila y una columna**
`UPDATE ... SET col = (subconsulta)`	**una fila y una columna**
`FROM (subconsulta) alias`	cualquier cantidad de filas y columnas, **con alias obligatorio**
`EXISTS (subconsulta)`	da igual: solo importa si hay alguna fila

🎙️ ¿Cómo te asegurás de que devuelva exactamente una fila y una columna? Con **una función de agregación sin GROUP BY**, que, como viste en el 4.5, siempre devuelve una fila (aunque sea 0 o NULL).

Subconsulta en el WHERE (ejemplo del profe: las facturas de clientes que no son consumidores finales):

```
SELECT * FROM Factura
WHERE fact_cliente NOT IN ( SELECT clie_codigo FROM Cliente
                            WHERE clie_razon_social LIKE 'CONSUMIDOR FINAL%' )
```

TOP. Es propio de T-SQL (en otros motores es LIMIT): `SELECT TOP 5 ...` devuelve las primeras 5 filas. Se ejecuta **después del ORDER BY**: primero se ordena y después se cortan las primeras. Combinado con IN, permite filtrar por un ranking:

```
WHERE c.clie_codigo IN ( SELECT TOP 10 fact_cliente FROM Factura
                         GROUP BY fact_cliente ORDER BY SUM(fact_total) DESC )
```

7.3 Subconsultas correlacionadas

Hasta acá las subconsultas eran **independientes**: podías seleccionarlas y ejecutarlas solas. Una subconsulta **correlacionada** usa una columna de la consulta de afuera, y por eso no se puede ejecutar sola.

🎙️ Primer ejemplo del profe: completar el total de las facturas que lo tienen vacío, sumando sus renglones.

```
UPDATE Factura
SET fact_total = ( SELECT SUM(i.item_cantidad * i.item_precio)
                   FROM Item_Factura i
                   WHERE i.item_tipo     = Factura.fact_tipo       -- ← columnas de afuera
                     AND i.item_sucursal = Factura.fact_sucursal
                     AND i.item_numero   = Factura.fact_numero )
WHERE fact_total IS NULL
```

Cómo se ejecuta: el WHERE de afuera deja N filas. Por cada una, el motor reemplaza los valores de afuera dentro de la subconsulta y la ejecuta, o sea, **N veces**, *"una por cada fila o, dicho de otra forma, una por cada valor distinto"*. Y el resultado se asigna a **esa** fila: se mantiene la correlación.

Segundo ejemplo, en el WHERE: el cliente con mayor límite de crédito de cada vendedor. Agrupando por vendedor no podés mostrar los datos del cliente (capítulo 4). Con una subconsulta correlacionada, sí:

```
SELECT *
FROM Cliente c1
WHERE c1.clie_limite_credito = ( SELECT MAX(c2.clie_limite_credito)
                                 FROM Cliente c2
                                 WHERE c2.clie_vendedor = c1.clie_vendedor )
ORDER BY c1.clie_vendedor
```

🎙️ Son dos *"fotos"* de la misma tabla. Para cada cliente de afuera (`c1`), la de adentro (`c2`) calcula el máximo **de su vendedor**. No compara uno contra todos. Si dos clientes empatan en el máximo, aparecen los dos.

7.4 EXISTS y NOT EXISTS

`EXISTS (subconsulta)` es verdadero si la subconsulta devuelve **al menos una fila**. No importa qué columnas devuelva: `SELECT 1` o `SELECT *` da lo mismo.

```
-- clientes que compraron en 2011 y NO volvieron a comprar en 2012
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
WHERE EXISTS     ( SELECT 1 FROM Factura f WHERE f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2011 )
  AND NOT EXISTS ( SELECT 1 FROM Factura f WHERE f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012 )
```

🎙️ EXISTS **no lleva nada a memoria**: es verdadero o falso y ahí termina, y se resuelve en disco. Es la forma natural de decir "tiene / no tiene", "alguna vez", "nunca".

⚠️ **La trampa del EXISTS con agregado.** Por la regla del 4.5, esto es **siempre verdadero**:

```
WHERE EXISTS ( SELECT COUNT(*) FROM Factura f WHERE f.fact_cliente = c.clie_codigo )
```

`COUNT(*)` sin GROUP BY siempre devuelve una fila (aunque valga 0), así que EXISTS siempre la encuentra. 🎙️ El profe lo llamó *"mal programado"*. Si querés una condición sobre un total, compará la subconsulta afuera (`WHERE (SELECT COUNT(*) ...) >= 5`) o usá HAVING adentro del EXISTS.

7.5 Subconsulta en el SELECT

Una subconsulta en el SELECT calcula **una columna** para cada fila del resultado:

```
SELECT f1.fact_cliente,
    SUM(i.item_cantidad) AS unidades_2011,
    ( SELECT SUM(f2.fact_total) FROM Factura f2
      WHERE f2.fact_cliente = f1.fact_cliente AND YEAR(f2.fact_fecha) = 2011 ) AS monto_2011
FROM Factura f1
JOIN Item_Factura i ON i.item_tipo = f1.fact_tipo AND i.item_sucursal = f1.fact_sucursal AND i.item_numero =
f1.fact_numero
WHERE YEAR(f1.fact_fecha) = 2011
GROUP BY f1.fact_cliente
```

🎙️ El profe preguntó por qué no poner directamente `SUM(fact_total)` en el SELECT principal. Por la multiplicación del capítulo 6: después de unir con los renglones, cada factura se repite una vez por renglón, y su total *"va acumulando"*. En el ejemplo de clase, una factura de 13,83 con 3 renglones se sumaba 3 veces. La subconsulta calcula el monto por su cuenta, sin la repetición.

7.6 Cuánto cuesta cada una

🎙️ El ranking del profe:

- **En el SELECT:** *"las más inocentes"*. Se ejecutan **al final**, sobre las filas que ya quedaron después del WHERE y del GROUP BY, y devuelven un solo valor.
- **En el FROM:** *"las más complicadas"*. Se ejecutan **al principio**, y todo su resultado va a memoria, sin índices, para que el resto de la consulta trabaje encima.
- **EXISTS:** no lleva nada a memoria.

Dos aclaraciones del profe. La correlacionada se ejecuta una vez por fila, así que conviene que lo de afuera sea chico (filtrá antes). Y reescribirla como JOIN no elimina ese trabajo: *"un join también busca uno por uno"*, aunque no lo veas en el código.

⚠️ En los parciales anteriores se prohibió usar subconsultas en el FROM y CTEs. En el capítulo 8 vas a ver que nunca hacen falta.

7.7 Vistas

Una **vista** es una consulta guardada con nombre, que después se usa como si fuera una tabla:

```
CREATE VIEW dbo.vw_stock_por_producto AS
SELECT p.prod_codigo, p.prod_detalle, SUM(s.stoc_cantidad) AS stock_total
FROM Producto p JOIN STOCK s ON s.stoc_producto = p.prod_codigo
GROUP BY p.prod_codigo, p.prod_detalle
GO
SELECT * FROM dbo.vw_stock_por_producto WHERE stock_total > 100
```

🎙️ Lo que remarcó el profe:

- La vista guarda **el código**, no el resultado. Cuando la usás, el motor reemplaza el nombre por su SELECT y optimiza todo junto. **No** la trata como una subconsulta en el FROM.
- **Performance: exactamente igual** que escribir la consulta a mano, *"ni para mejor ni para peor"*. ⚠️ Si en el parcial justificás una vista "para que sea más rápido", está mal.
- Sirve para dos cosas:
  - **Simplificar:** no repetir siempre la misma condición, por ejemplo "solo productos activos".
  - **Seguridad:** podés darle permiso a un usuario sobre la vista sin dárselo sobre la tabla. Además, la vista puede ocultar columnas (por ejemplo, la clave encriptada de los usuarios) o renombrarlas (ocultar la metadata).
- **No admite ORDER BY** adentro: el orden tiene que ser lo último que se ejecuta, y va en la consulta que usa la vista. (SQL Server lo acepta si la vista tiene TOP, pero eso no garantiza el orden de afuera.)
- Se puede hacer INSERT, UPDATE o DELETE a través de una vista si es de **una sola tabla, sin agrupar**, y si las columnas que deja afuera aceptan NULL o tienen valor por defecto.
- `WITH CHECK OPTION` rechaza las modificaciones que dejarían filas que la vista no mostraría. El ejemplo del profe: una vista de clientes con `fecha_alta IS NOT NULL`. Sin esta opción, una aplicación inserta un cliente sin fecha a través de la vista, muestra "operación exitosa" en verde... y el cliente no aparece en la vista. 🎙️ *"No es obligatorio, pero sí medio como muy necesario."*

Existen también las **vistas indexadas**, que sí guardan el resultado. Se ven en el capítulo 12, porque su uso es justamente "tener datos precalculados".

7.8 Operadores de conjunto: apilar resultados

Hasta acá, para combinar dos tablas usaste JOIN, que las pega **de costado**: el resultado tiene las columnas de las dos y solo las filas que se emparejan. Los **operadores de conjunto** combinan en la otra dirección: toman el resultado de dos SELECT completos y los **apilan uno debajo del otro**. El resultado tiene las columnas de un solo SELECT y las filas de los dos, según la regla del operador.

Son tres, los mismos de la teoría de conjuntos:

Operador	Devuelve	Diagrama
`UNION`	lo que está en A **o** en B	todo
`INTERSECT`	lo que está en A **y** en B	la parte común
`EXCEPT`	lo que está en A y **no** en B	A menos B (en Oracle se escribe `MINUS`)

Las reglas para escribirlos:

1. **Cada SELECT es una consulta completa** por su cuenta, con su FROM, su WHERE, su GROUP BY y su HAVING.
2. Los dos tienen que devolver **la misma cantidad de columnas**, en el mismo orden y con tipos compatibles. Los **nombres** de las columnas del resultado salen del **primer** SELECT.
3. Se compara **la fila entera**: dos filas son "la misma" solo si coinciden en todas las columnas.
4. Si a una de las partes le falta una columna, se completa con un valor fijo: `NULL`, `0` o un texto.

Veámoslo con dos conjuntos chicos escritos a mano. A tiene 10, 11 y 12; B tiene 11, 12, 12 y 13:

```
SELECT v FROM (VALUES (10), (11), (12)) AS a(v)
UNION
SELECT v FROM (VALUES (11), (12), (12), (13)) AS b(v)
```

Operador	Resultado
A UNION B	10, 11, 12, 13
A UNION ALL B	10, 11, 12, 11, 12, 12, 13 (7 filas)
A INTERSECT B	11, 12
A EXCEPT B	10
B EXCEPT A	13

⚠️ **Una corrección a lo que se dijo en clase.** 🎙️ El profe explicó que UNION elimina los duplicados "entre un conjunto y otro" y que por eso el 12, que está dos veces en B, aparecería dos veces en el resultado. **No es así**: probado en tu base, el 12 aparece **una sola vez**. `UNION`, `INTERSECT` y `EXCEPT` devuelven el resultado **sin ninguna fila repetida**, como si le aplicaras un DISTINCT al final, vengan las repetidas de donde vengan. El único que conserva todo es `UNION ALL`.

UNION vs UNION ALL. `UNION ALL` no elimina nada: pega las filas de las dos partes tal cual. 🎙️ El profe lo recomienda **cuando sabés que no puede haber repetidos**, porque le ahorrás al motor el trabajo de comparar todas las filas buscando duplicados. Por ejemplo, si una parte trae los clientes que compraron y la otra los que no compraron, ningún cliente puede estar en las dos.

El orden de las partes. En `UNION` e `INTERSECT` da igual cuál va primero, y podés encadenar tres o más SELECT. En `EXCEPT` el orden cambia el resultado (A − B no es B − A). 🎙️ El profe dijo que EXCEPT admite solo dos conjuntos; en realidad SQL Server te deja encadenar más y los resuelve de izquierda a derecha, pero cuando mezcles operadores **usá paréntesis**: `INTERSECT` se resuelve antes que `UNION` y `EXCEPT`, y es fácil equivocarse.

Con la base

¿Qué clientes compraron en 2011 **y también** en 2012?

```
SELECT fact_cliente FROM Factura WHERE YEAR(fact_fecha) = 2011
INTERSECT
SELECT fact_cliente FROM Factura WHERE YEAR(fact_fecha) = 2012
ORDER BY 1
```

Un cliente con 50 facturas aparece una sola vez, sin DISTINCT: el operador ya saca los repetidos. En tu base, 39 clientes compraron en 2011; con `INTERSECT` quedan **36** (compraron los dos años) y cambiando a `EXCEPT` quedan **3** (compraron en 2011 y en 2012 no). Las cuentas cierran: 36 + 3 = 39.

¿Hacía falta un operador de conjunto? No: `INTERSECT` se puede escribir con `EXISTS`, `EXCEPT` con `NOT EXISTS`, y una `UNION` de la misma tabla con un `OR` en el WHERE. Donde el operador **sí** simplifica es cuando cada parte necesita **su propia agrupación**. Por ejemplo, los clientes que en 2011 tuvieron al menos 15 facturas **o** en 2012 al menos 30:

```
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2011
GROUP BY c.clie_codigo, c.clie_razon_social
HAVING COUNT(*) >= 15
UNION
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY c.clie_codigo, c.clie_razon_social
HAVING COUNT(*) >= 30
ORDER BY 2
```

Devuelve 29 clientes. Acá va `UNION` y no `UNION ALL`, porque un cliente puede cumplir las dos condiciones y lo queremos una sola vez.

Las partes también pueden venir de **tablas distintas**. Una agenda con clientes y empleados juntos:

```
SELECT clie_razon_social AS nombre, clie_telefono AS telefono, 'Cliente' AS tipo
FROM Cliente
UNION ALL
SELECT RTRIM(empl_apellido) + ', ' + RTRIM(empl_nombre), NULL, 'Empleado'   -- Empleado no tiene teléfono: NULL
FROM Empleado
ORDER BY nombre
```

Los nombres de las columnas (`nombre`, `telefono`, `tipo`) los pone el primer SELECT; el segundo no necesita alias.

Un solo ORDER BY, al final

En los ejemplos de arriba el `ORDER BY` parece pertenecer al último SELECT, pero **ordena el resultado completo**. 🎙️ *"El ordenamiento es cuando ya sabemos los datos que tenemos que mostrar"*: el motor resuelve cada SELECT, aplica el operador y recién después ordena. Por eso hay **un solo** ORDER BY, va **al final**, y usa los nombres del primer SELECT o el número de columna.

⚠️ **TOP en las partes.** Supongamos que querés los 5 primeros clientes del alfabeto y los 5 últimos:

```
-- NO COMPILA: "Incorrect syntax near the keyword 'UNION'"
SELECT TOP 5 clie_razon_social FROM Cliente ORDER BY clie_razon_social
UNION
SELECT TOP 5 clie_razon_social FROM Cliente ORDER BY clie_razon_social DESC
```

🎙️ El profe lo explicó como una inconsistencia: el TOP necesita su ORDER BY (es lo único que se ejecuta *después* de ordenar), pero en un operador de conjunto el ORDER BY va después del operador. Las dos reglas no pueden cumplirse a la vez. La salida es encerrar cada TOP en una subconsulta, que tiene su propio orden:

```
SELECT clie_razon_social FROM (SELECT TOP 5 clie_razon_social FROM Cliente
                               ORDER BY clie_razon_social) AS primeros
UNION
SELECT clie_razon_social FROM (SELECT TOP 5 clie_razon_social FROM Cliente
                               ORDER BY clie_razon_social DESC) AS ultimos
ORDER BY clie_razon_social
```

Si el enunciado prohíbe subconsultas en el FROM, el TOP va adentro de un `IN`, como en la receta 11 del capítulo 8.

Una tabla que no existe: los 12 meses

Una UNION de valores fijos, sin ninguna tabla, arma un conjunto que podés usar como tabla. 🎙️ El profe la usó para el clásico "ventas por mes, **incluidos los meses sin ventas**":

```
SELECT m.mes,
       COUNT(f.fact_numero)       AS facturas,
       ISNULL(SUM(f.fact_total), 0) AS total
FROM (SELECT 1 AS mes UNION ALL SELECT 2 UNION ALL SELECT 3 UNION ALL SELECT 4
      UNION ALL SELECT 5 UNION ALL SELECT 6 UNION ALL SELECT 7 UNION ALL SELECT 8
      UNION ALL SELECT 9 UNION ALL SELECT 10 UNION ALL SELECT 11 UNION ALL SELECT 12) AS m
LEFT JOIN Factura f ON MONTH(f.fact_fecha) = m.mes
                   AND YEAR(f.fact_fecha) = 2011        -- filtro de la dominada: en el ON (6.4)
GROUP BY m.mes
ORDER BY m.mes
```

La tabla de meses es la **dominante**: tiene siempre 12 filas, así que los meses sin facturas aparecen igual, con 0. En tu base, 2011 tiene ventas recién desde julio: de enero a junio sale 0, y diciembre tiene 200 facturas por 55568.79. Fijate también que el ON no compara una clave foránea: un ON acepta **cualquier condición**, y acá la condición es "el mes de la factura es este mes".

(En SQL Server hay una forma más corta de escribir la misma tabla: `FROM (VALUES (1), (2), ..., (12)) AS m(mes)`.)

7.9 Numerar filas: ROW_NUMBER

A veces el resultado necesita un número de orden: 🎙️ *"vamos a imprimir un acta, una lista de asistencia"*, y el primero de la lista es el 1, el siguiente el 2, y así. Eso hace `ROW_NUMBER()`:

```
SELECT ROW_NUMBER() OVER (ORDER BY prod_familia, prod_codigo) AS orden,
       prod_codigo, prod_detalle, prod_familia
FROM Producto
```

El `ORDER BY` de adentro del `OVER` es obligatorio: dice en qué orden se reparten los números, y es independiente del ORDER BY final de la consulta. En tu base numera los 2190 productos.

⚠️ **No se puede filtrar por el número en el WHERE.** Si querés las filas 10 a 15, `WHERE orden BETWEEN 10 AND 15` da *"Invalid column name 'orden'"*. Es el orden lógico del 4.4: el WHERE se resuelve antes que el SELECT, así que el alias todavía no existe (y aunque escribas la expresión entera, las funciones de ventana no están permitidas en el WHERE). La salida es envolver la consulta en una subconsulta en el FROM, que como toda tabla del FROM necesita un nombre:

```
SELECT *
FROM (SELECT ROW_NUMBER() OVER (ORDER BY prod_familia, prod_codigo) AS orden,
             prod_codigo, prod_detalle, prod_familia
      FROM Producto) AS ss
WHERE ss.orden BETWEEN 10 AND 15
```

Numerar por grupo: PARTITION BY. Agregando `PARTITION BY`, la numeración vuelve a empezar en 1 cada vez que cambia el grupo (lo que en programación se llama un **corte de control**):

```
SELECT prod_familia,
    ROW_NUMBER() OVER (PARTITION BY prod_familia ORDER BY prod_codigo) AS nro,
    prod_codigo, prod_detalle
FROM Producto
ORDER BY prod_familia, prod_codigo
```

La familia 001 numera 1 y 2, la 002 vuelve a empezar (1, 2, 3, 4), la 003 empieza otra vez de 1, y así. A diferencia del GROUP BY, **no junta filas**: cada producto sigue siendo una fila, solo que con su número dentro del grupo. La columna puede ir en cualquier lugar del SELECT, no tiene que ser la primera.

Para el parcial: si te prohíben subconsultas en el FROM, no vas a poder filtrar por el número. Problemas como "el primero de cada grupo" se resuelven igual con un mandadero TOP 1 (receta 1 del capítulo 8).

Para llevarte

- IN: una columna. `=`: un valor. SELECT y SET: una fila y una columna (agregado sin GROUP BY).
- Correlacionada = se ejecuta una vez por fila de afuera.
- `EXISTS (SELECT COUNT(*) ...)` es siempre verdadero.
- Más barata la del SELECT; más cara la del FROM.
- Vista = consulta guardada. Sirve para simplificar y para seguridad, **nunca** para performance.
- UNION / INTERSECT / EXCEPT **apilan** dos SELECT con las mismas columnas y devuelven el resultado **sin repetidos**. `UNION ALL` deja todo y es más barato: usalo cuando sabés que no hay repetidos.
- Un solo ORDER BY, al final, para todo el resultado. TOP en una de las partes → subconsulta.
- `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` numera sin agrupar. No va en el WHERE.

Capítulo 8 — Resolver consultas de parcial

Con los capítulos 2 a 7 ya tenés todas las piezas: filtros, agrupación, JOINs y subconsultas. Lo que falta es **saber cuál usar** cuando el enunciado mezcla varias cosas, como pasa en el parcial ("para cada producto, mostrar... solo los que en dos años consecutivos... ordenados primero por..."). Este capítulo te da un método para desarmar cualquier enunciado y un recetario de patrones que se repiten. Cada receta trae un ejercicio resuelto y probado contra tu base.

La idea que une todo el capítulo viene del 6.3 y el 7.5: **cuando una tabla repite filas, las sumas se rompen; la salida es calcular aparte, con una subconsulta**.

8.1 Dónde vive cada dato

Antes de empezar, un repaso del capítulo 1 aplicado a las preguntas más comunes. Esto evita la mitad de los errores:

- ¿**Unidades** vendidas? → `SUM(item_cantidad)` (está en el renglón).
- ¿**Monto** vendido? → `SUM(fact_total)` si es por factura, o `SUM(item_cantidad * item_precio)` si es por producto (¡no sumes `fact_total` agrupando por producto: el ticket se repetiría por cada renglón!).
- ¿**Fecha / cliente / vendedor**? → están en `Factura`, no en el renglón. Si pensás "por producto" y necesitás fecha, siempre vas a tener que unir con `Factura`.
- ¿Un producto **es compuesto**? → `prod_codigo IN (SELECT comp_producto FROM Composicion)`.
- ¿**Cuántos depósitos** hay? → `(SELECT COUNT(*) FROM DEPOSITO)`.

8.2 El método: la planilla y los mandaderos

Esta es la idea central de todo el parcial de consultas.

La metáfora

- La **consulta principal** es una **planilla**: cada fila es *un sujeto* (un cliente, un producto, un año-producto...).
- Cada **columna** que te piden es una **pregunta sobre ese sujeto**.
- Si la respuesta sale "de lo que ya tenés en la mesa" (el mismo FROM, el mismo filtro), es un agregado normal: `SUM`, `COUNT`, `MAX`.
- Si la respuesta necesita **otro filtro, otro año, otra tabla hija**, mandás a un **mandadero**: una **subconsulta correlacionada en el SELECT**. El mandadero lleva en la mano el código de la fila (`WHERE x.cliente = c.clie_codigo`), va, busca, y vuelve con **un solo valor**.

Los 5 pasos (siempre en este orden)

Paso 1 — ¿Qué es UNA fila del resultado? (el sujeto) Leé lo que piden mostrar y preguntate "¿una fila por qué cosa?".

- "Mostrar para cada cliente..." → `FROM Cliente c`
- "Por producto..." → `FROM Producto p`

- "Por año y producto..." → `FROM Item_Factura JOIN Factura ... GROUP BY YEAR(fecha), producto`

> Regla de oro: **arrancá el FROM desde la tabla del sujeto**, aunque los datos estén en otro lado. Es más fácil (y más rápido, `Producto` tiene 2190 filas e `Item_Factura` cientos de miles).

Paso 2 — Cada columna: ¿mesa o mandadero? Para cada columna pedida, preguntate: *¿sale del mismo FROM con el mismo filtro?*

- Sí → agregado normal.
- No (otro año, otra tabla hija, "el que más...") → subconsulta en el SELECT.

Paso 3 — Filtros: ¿filas, grupos o existencia?

- Condición sobre **una fila** (año, rubro) → `WHERE`.
- Condición sobre **un total del grupo** ("vendió más de 100") → `HAVING`.
- Condición tipo **"tiene/no tiene", "existe un año en que...", "alguna vez"** → `WHERE EXISTS (...)` / `NOT EXISTS`.
- Condición tipo **"está entre los 10 que..."** → `WHERE x IN (SELECT TOP 10 ...)`.

Paso 4 — Orden y TOP. `ORDER BY` puede llevar un `CASE` o una subconsulta ("primero los que..., después el resto").

Paso 5 — Chequeo de cardinalidad (el detector de humo). Preguntate: *¿algún JOIN multiplica filas antes de un SUM/AVG/COUNT?*

- Si unís el sujeto con **dos tablas hijas distintas** (ej: producto con `Item_Factura` **y** con `STOCK`), cada renglón se repite por cada depósito → los `SUM` salen inflados.
- Solución: **una tabla hija en el FROM, la otra por mandadero** (subconsulta).
- Excepciones que no se rompen con la multiplicación: `COUNT(DISTINCT ...)`, `MAX`, `MIN`.
- 🎙️ Así lo explicó el profe: "lo que se rompe con la multiplicación son las **sumatorias, los promedios y los count**; el mínimo y el máximo no". Su ejemplo (Ej. 7): unir producto con `STOCK` **y** con `Item_Factura` está bien si pedís `MIN` / `MAX` del precio, y está mal si pedís sumas. Y en la clase de subselect: `SUM(fact_total)` unido a los renglones suma el total del ticket una vez por renglón → mandadero.

Plantilla universal

```
SELECT
  s.codigo,
  s.detalle,
  AGREGADO(...)                          AS col_de_la_mesa,     -- paso 2: sale del FROM
  ( SELECT ... WHERE x.fk = s.codigo AND ... )   AS col_mandadero      -- paso 2: otro filtro / otra tabla
FROM TablaDelSujeto s
[JOIN tabla_hija_principal ...]
WHERE  condicion_por_fila                                    -- paso 3
 AND  [NOT] EXISTS ( ... correlacionada con s ... )
 AND  s.codigo IN ( SELECT TOP N ... )
GROUP BY s.codigo, s.detalle
HAVING  condicion_sobre_el_grupo
ORDER BY ...                                                 -- paso 4
```

8.3 Self-joins con método

En el 6.5 viste qué es un self-join. Acá va cómo armarlo sin perderte. El truco mental es el mismo: **no es "la tabla con ella misma", son DOS ROLES distintos que casualmente viven en la misma tabla.**

Workflow de 5 pasos

1. **Nombrá los roles en castellano** antes de escribir SQL. "Necesito un *empleado* y su *jefe*". "Necesito un *producto A* y un *producto B* que estén en el mismo ticket". "Necesito *este año* y *el año siguiente*".
2. **Alias = rol**, nunca `t1` / `t2`: `e` y `j`, `ia` e `ib`, `anio` y `sig`. Con alias con sentido no te perdés en el SELECT.
3. **El ON es la frase que une los roles**, traducida:
   - "el jefe del empleado" → `j.empl_codigo = e.empl_jefe`
   - "en el mismo ticket" → `ib.item_tipo = ia.item_tipo AND ib.item_sucursal = ia.item_sucursal AND ib.item_numero = ia.item_numero`
4. **Decidí la simetría** (solo cuando los roles son intercambiables, como "pares"):
   - Sin nada: aparece (A,A) → mal.
   - `<>`: aparece (A,B) **y** (B,A) → pares duplicados.
   - `>` (o `<`): cada par una sola vez. **"Sin repetir pares" = `>`**.
   - En jerarquías (empleado/jefe) no hace falta: los roles no son intercambiables.
5. **Chequeá la multiplicación**: un self-join de renglones genera *todas las combinaciones*. `COUNT(*)` cuenta pares (bien si querés "veces juntos"), pero un `SUM(item_cantidad)` de un lado sale multiplicado por la cantidad de pareja que tuvo. Si necesitás sumar, usá mandadero.

Los 3 sabores de self-join (si ves A → B)

Si el enunciado dice...	Es el sabor...	Cómo se arma
"jefe", "a cargo de", "empleado que lo tiene como jefe"	**Jerarquía**	`Empleado e JOIN Empleado j ON j.empl_codigo = e.empl_jefe`
"combo de componentes" (con datos de los dos productos)	**Jerarquía** sobre `Producto` vía `Composicion`	`Composicion c JOIN Producto combo ON combo.prod_codigo = c.comp_producto JOIN Producto comp ON comp.prod_codigo = c.comp_componente`
"pares", "vendidos juntos", "en la misma factura"	**Pares**	`Item_Factura ia JOIN Item_Factura ib ON (misma factura) AND ib.item_producto > ia.item_producto`
"años consecutivos", "respecto al año anterior", "mismo mes del año anterior"	**Temporal**	⚠️ **No uses self-join de tablas crudas**: usá el **año ancla** (receta 4).

> **¿Por qué no self-join para lo temporal?** Porque lo que querés comparar no son *filas*, son *totales por año*. Un `Factura f1 JOIN Factura f2 ON YEAR(f2)=YEAR(f1)+1` cruza cada factura de un año con cada factura del siguiente: millones de filas y SUMs multiplicados. Sin subconsultas en el FROM no podés "pre-agrupar", así que la salida limpia es el año ancla.

Ejemplo modelo (Ej. 15, pares)

🎙️ Así lo corrigió el profe en clase: **no hace falta unir con `Factura`**. Se igualan los dos renglones por las tres columnas (tipo, sucursal, número) y con `>` se queda una sola combinación de cada par. También dijo que la consulta tarda (genera muchas combinaciones), así que conviene probarla con un umbral más chico antes de poner el `> 500`.

```
SELECT ia.item_producto, pa.prod_detalle, ib.item_producto, pb.prod_detalle, COUNT(*) AS veces_juntos
FROM Item_Factura ia                          -- rol: producto A
JOIN Item_Factura ib                          -- rol: producto B
  ON  ib.item_tipo     = ia.item_tipo         -- "en el mismo ticket"
  AND ib.item_sucursal = ia.item_sucursal
  AND ib.item_numero   = ia.item_numero
  AND ib.item_producto > ia.item_producto     -- "sin repetir pares"
JOIN Producto pa ON pa.prod_codigo = ia.item_producto
JOIN Producto pb ON pb.prod_codigo = ib.item_producto
GROUP BY ia.item_producto, pa.prod_detalle, ib.item_producto, pb.prod_detalle
HAVING COUNT(*) > 500
ORDER BY 5 DESC                               -- 5 = veces_juntos (el profe prefiere número de columna antes que alias)
```

8.4 El recetario: si ves A → hacé B

Cada receta empieza con la señal que la dispara en el enunciado ("si ves...") y el molde de código. Después viene un ejercicio resuelto: leelo con el molde al lado.

Receta 1 — "El que más..." (el cliente que más compró, el producto más vendido)

Si ves "el X que más/menos..." para cada fila → **mandadero con TOP 1 + GROUP BY + ORDER BY agregado**.

```
( SELECT TOP 1 f2.fact_cliente
  FROM Item_Factura i2
  JOIN Factura f2 ON (clave de 3)
  WHERE i2.item_producto = p.prod_codigo      -- correlación con la fila de afuera
  GROUP BY f2.fact_cliente                    -- un grupo por candidato
  ORDER BY SUM(i2.item_cantidad) DESC,        -- "el que más"
        f2.fact_cliente ) AS cliente_top      -- desempate (si dice "el de menor código")
```

Metáfora: el mandadero arma un mini-ranking y te trae solo el primero. **Usá siempre el TOP 1 en el SELECT**, aunque el enunciado no prohíba las subconsultas en el FROM. 🎙️ El profe dijo que las subconsultas en el FROM son "las más complicadas": se ejecutan al principio, todo su resultado queda en memoria y ahí no hay índices que ayuden. En cambio, las del SELECT son "las más inocentes", porque corren al final, sobre las filas que ya quedaron después del WHERE/GROUP BY. Por eso una alternativa con `ROW_NUMBER()` dentro de un subselect en el FROM funciona, pero es justo lo que él desaconseja (y en el parcial del 20-09 directamente estaba prohibida).

✍️ Ejercicio resuelto 1

> **Enunciado:** Para cada rubro, mostrar código, detalle, cantidad de productos del rubro y el código del producto más vendido (en unidades) de ese rubro. Si hay empate, el de menor código.

Cómo lo pienso:

1. *Sujeto:* una fila por rubro → `FROM Rubro`.
2. *Cantidad de productos:* sale de la mesa si uno `Rubro` con `Producto` → `COUNT(*)`.
3. *El producto más vendido:* necesita otra tabla (`Item_Factura`) y un ranking → **mandadero** **`TOP 1`**. Adentro agrupo por producto (un grupo por candidato) y ordeno por unidades.
4. *Cardinalidad:* si metiera `Item_Factura` en el FROM de afuera, el `COUNT(*)` contaría renglones, no productos. Por eso las ventas van en el mandadero.

```
SELECT r.rubr_id, r.rubr_detalle,
    COUNT(*) AS cant_productos,                    -- mesa
    ( SELECT TOP 1 i.item_producto                 -- mandadero
      FROM Item_Factura i
      JOIN Producto p2 ON p2.prod_codigo = i.item_producto
      WHERE p2.prod_rubro = r.rubr_id              -- correlación con el rubro de afuera
      GROUP BY i.item_producto                     -- un grupo por candidato
      ORDER BY SUM(i.item_cantidad) DESC, i.item_producto   -- el que más; desempate: menor código
    ) AS producto_mas_vendido
FROM Rubro r
JOIN Producto p ON p.prod_rubro = r.rubr_id
GROUP BY r.rubr_id, r.rubr_detalle
ORDER BY 1
```

Resultado en tu base: 31 rubros (ej. CIGARRILLOS → `00001705`, PILAS → `00001420`). HACIENDAS y ESPECIALES muestran `NULL`: sus productos nunca se vendieron, así que el mandadero no encontró ninguna fila y vuelve con las manos vacías.

Receta 2 — "Los N primeros que cumplen..., y de ellos solo los que..."

Si ves dos filtros en secuencia ("de los seleccionados, mostrar únicamente...") → **el TOP va adentro de un `IN`, y el segundo filtro afuera**.

```
WHERE p.prod_codigo IN ( SELECT TOP 10 ... ORDER BY ... )   -- 1ro: elijo los 10
  AND (segundo filtro)                                      -- 2do: de esos 10, filtro
```

Si ponés los dos filtros juntos y el `TOP 10` afuera, te quedan "10 que cumplen ambas cosas", que es **otra** consulta. El orden de los filtros cambia el resultado: leé el enunciado como una receta de cocina.

✍️ Ejercicio resuelto 2

> **Enunciado:** De los 10 clientes que más facturaron en 2012 (suma de `fact_total`), mostrar únicamente los que tienen un límite de crédito menor a 4000. Mostrar código, razón social y límite de crédito.

Cómo lo pienso:

1. *Sujeto:* cliente → `FROM Cliente`.
2. *Primer filtro (elegir):* "los 10 que más facturaron en 2012" → `IN (SELECT TOP 10 ...)`. El ranking se arma **solo** con facturación, sin mirar el límite.
3. *Segundo filtro (de esos 10):* "límite menor a 4000" → va **afuera**, en el WHERE de la consulta principal.

```
SELECT c.clie_codigo, c.clie_razon_social, c.clie_limite_credito
FROM Cliente c
WHERE c.clie_codigo IN ( SELECT TOP 10 f.fact_cliente        -- 1ro: elijo los 10
                         FROM Factura f
                         WHERE YEAR(f.fact_fecha) = 2012
                         GROUP BY f.fact_cliente
                         ORDER BY SUM(f.fact_total) DESC, f.fact_cliente )
  AND c.clie_limite_credito < 4000                           -- 2do: de esos 10, filtro
ORDER BY 1
```

Resultado en tu base: 7 clientes (`01742`, `01743`, `01746`, `01747`, `01748`, `02088`, `03342`). Los otros 3 del top 10 tienen límite mayor o igual a 4000.

La versión incorrecta, con los dos filtros juntos y el TOP afuera (`WHERE YEAR(...) = 2012 AND c.clie_limite_credito < 4000` + `TOP 10`), devuelve 10 clientes, entre ellos `02269`, `01719` y `01745`, que **no** están entre los 10 que más facturaron. Responde otra pregunta: "los 10 que más facturaron *entre los de límite bajo*".

Receta 3 — "En todos los..." / "en al menos el X% de..."

Si ves "en todos los depósitos" → contá y compará contra el total:

```
HAVING COUNT(DISTINCT s.stoc_deposito) = ( SELECT COUNT(*) FROM DEPOSITO )
```

Si ves "en al menos el 75%" →

```
( SELECT COUNT(*) FROM STOCK s WHERE s.stoc_producto = p.prod_codigo AND s.stoc_cantidad > 0 )
   >= 0.75 * ( SELECT COUNT(*) FROM DEPOSITO )
```

Ojo: "dispone de stock" = `stoc_cantidad > 0`, no solo "tener fila en STOCK".

✍️ Ejercicio resuelto 3

> **Enunciado (a):** Mostrar los clientes que compraron en **todos** los años en los que hubo facturación.
>
> **Enunciado (b):** Mostrar los productos que tienen stock en **al menos el 35%** de los depósitos. Mostrar código, detalle y en cuántos depósitos tiene stock.

Cómo lo pienso (a): "todos los años" = la cantidad de años distintos en que compró el cliente es igual a la cantidad de años distintos que hay en `Factura`. El total va en una subconsulta: **no escribas `= 3` a mano**, porque si se agrega un año la consulta deja de estar bien.

```
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
JOIN Factura f ON f.fact_cliente = c.clie_codigo
GROUP BY c.clie_codigo, c.clie_razon_social
HAVING COUNT(DISTINCT YEAR(f.fact_fecha))                  -- años en que compró ESTE cliente
   = ( SELECT COUNT(DISTINCT YEAR(fact_fecha)) FROM Factura )   -- años que existen
```

Resultado en tu base: 1 cliente, `01772` ROSAS LUIS MIGUEL (el único que compró en 2010, 2011 y 2012).

Cómo lo pienso (b): sujeto = producto. La cantidad de depósitos con stock es un mandadero, y la uso dos veces: para mostrarla y para filtrar. "Tiene stock" = `stoc_cantidad > 0`.

```
SELECT p.prod_codigo, p.prod_detalle,
     ( SELECT COUNT(*) FROM STOCK s
       WHERE s.stoc_producto = p.prod_codigo AND s.stoc_cantidad > 0 ) AS depositos_con_stock
FROM Producto p
WHERE ( SELECT COUNT(*) FROM STOCK s
      WHERE s.stoc_producto = p.prod_codigo AND s.stoc_cantidad > 0 )
    >= 0.35 * ( SELECT COUNT(*) FROM DEPOSITO )            -- 0.35 * 33 = 11.55 → hacen falta 12
ORDER BY 3 DESC, 1
```

Resultado en tu base: 12 productos, todos con stock en 12 depósitos (casi todos cigarrillos). Con `0.30` aparecen 44.

Receta 4 — Comparar años: CASE (años fijos) vs. AÑO ANCLA (años genéricos)

Si ves años con nombre ("2012 vs 2011") → **truco del CASE** (pivot): una sola pasada, una columna por año.

```
SUM(CASE WHEN YEAR(f.fact_fecha) = 2012 THEN i.item_cantidad ELSE 0 END) AS c2012
...
HAVING SUM(CASE ... 2012 ...) > SUM(CASE ... 2011 ...)
```

Si ves años genéricos ("dos años consecutivos", "respecto del año anterior", "en algún momento") → **AÑO ANCLA**:

> Metáfora: clavás una estaca en un año (Y) y mandás mandaderos a Y+1, Y+2. Si para **alguna** estaca se cumple, el sujeto entra.

```
WHERE EXISTS (
   SELECT 1
   FROM Factura f1 JOIN Item_Factura i1 ON (clave de 3)
   WHERE f1.fact_cliente = c.clie_codigo
   GROUP BY YEAR(f1.fact_fecha)                    -- cada grupo = una estaca (año Y)
   HAVING (condición en Y, con agregados de f1/i1)
     AND ( SELECT ... WHERE ... AND YEAR(f2.fact_fecha) = YEAR(f1.fact_fecha) + 1 ) (condición en Y+1)
)
```

Dentro del `HAVING` podés usar `YEAR(f1.fact_fecha)` porque es lo que agrupaste. Para 3 años: otro mandadero a `+ 2`.

✍️ Ejercicio resuelto 4 (a) — años fijos, truco del CASE

> **Enunciado:** Para los productos del rubro `0011` (LINTERNAS), mostrar los que vendieron más unidades en 2012 que en 2011. Mostrar código, detalle, unidades 2011 y unidades 2012.

Cómo lo pienso: los años tienen nombre (2011 y 2012) → una sola pasada por los renglones y **una columna por año** con `SUM(CASE ...)`. La comparación entre años es sobre totales → `HAVING`. El `ELSE 0` hace que un producto sin ventas en 2011 muestre 0 y no NULL.

```
SELECT p.prod_codigo, p.prod_detalle,
     SUM(CASE WHEN YEAR(f.fact_fecha) = 2011 THEN i.item_cantidad ELSE 0 END) AS unidades_2011,
     SUM(CASE WHEN YEAR(f.fact_fecha) = 2012 THEN i.item_cantidad ELSE 0 END) AS unidades_2012
FROM Producto p
JOIN Item_Factura i ON i.item_producto = p.prod_codigo
JOIN Factura f ON f.fact_tipo = i.item_tipo AND f.fact_sucursal = i.item_sucursal AND f.fact_numero = i.item_numero
WHERE p.prod_rubro = '0011'                         -- filtro por fila → WHERE
GROUP BY p.prod_codigo, p.prod_detalle
HAVING SUM(CASE WHEN YEAR(f.fact_fecha) = 2012 THEN i.item_cantidad ELSE 0 END)
   > SUM(CASE WHEN YEAR(f.fact_fecha) = 2011 THEN i.item_cantidad ELSE 0 END)   -- filtro por total → HAVING
ORDER BY 1
```

Resultado en tu base: 11 productos (ej. `00001718` "Linterna con pilas": 2265 en 2011 → 28800 en 2012).

✍️ Ejercicio resuelto 4 (b) — años genéricos, año ancla

> **Enunciado:** Mostrar los clientes que **en algún año** compraron (en monto) más del doble que el año anterior.

Cómo lo pienso: "en algún año" y "el anterior" no nombran años → **año ancla**. Clavo la estaca en el año Y (agrupando las facturas del cliente por año) y mando un mandadero a Y+1. Si para alguna estaca se cumple, el `EXISTS` da verdadero.

```sql
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
WHERE EXISTS (
  SELECT 1
  FROM Factura f1
  WHERE f1.fact_cliente = c.clie_codigo
  GROUP BY YEAR(f1.fact_fecha)                    -- cada grupo = una estaca (año Y)
  HAVING ( SELECT SUM(f2.fact_total)              -- mandadero al año Y+1
        FROM Factura f2
        WHERE f2.fact_cliente = c.clie_codigo
          AND YEAR(f2.fact_fecha) = YEAR(f1.fact_fecha) + 1 )
       > 2 * SUM(f1.fact_total)                   -- monto del año Y
)
ORDER BY 1
```

Resultado en tu base: 33 clientes. Dos detalles:

- Si el cliente no compró en Y+1, el mandadero devuelve NULL y `NULL > x` es falso: esa estaca no cuenta, que es lo que queremos.
- **Supuesto para escribir en el parcial:** un cliente que no compró nada en Y pero sí en Y+1 no entra, porque sin compras en Y no hay estaca. Si el profe quisiera contarlo ("de 0 a algo es más del doble"), habría que anclar desde Y+1 y mirar hacia atrás con `ISNULL(..., 0)`.

Receta 5 — "Solo...", "ninguno...", "todos sus items son..." (doble negación)

Si ves "compró SOLO productos compuestos" → SQL no tiene "para todo", así que se dice al revés: **"no existe ninguno que no sea compuesto"**.

```sql
NOT EXISTS ( SELECT 1 FROM Item_Factura i
        WHERE (i es de esta factura)
          AND i.item_producto NOT IN ( SELECT comp_producto FROM Composicion ) )
```

Metáfora: para decir "todos los invitados tienen entrada", el patovica busca a alguien *sin* entrada; si no encuentra a nadie, se cumple.

✍️ Ejercicio resuelto 5

> **Enunciado:** Mostrar los depósitos en los que **todos** los productos cargados tienen stock positivo (ninguna fila con cantidad 0 o negativa). Mostrar código, detalle y cuántos productos tiene cargados.

Cómo lo pienso:

1. "Todos tienen stock positivo" se dice al revés: **no existe** ninguna fila de ese depósito con `stoc_cantidad <= 0` → `NOT EXISTS`.
2. **La trampa del "todos" vacío:** un depósito sin ninguna fila en `STOCK` también cumple el `NOT EXISTS` (el patovica no encuentra a nadie sin entrada... porque no vino nadie). Para que no aparezcan, exijo que tenga al menos un producto: el `JOIN STOCK` lo garantiza, y además me da la cantidad de productos.

```sql
SELECT d.depo_codigo, d.depo_detalle, COUNT(*) AS productos
FROM DEPOSITO d
JOIN STOCK s ON s.stoc_deposito = d.depo_codigo          -- tiene al menos un producto
WHERE NOT EXISTS ( SELECT 1 FROM STOCK s2                -- y no hay ninguno "malo"
                   WHERE s2.stoc_deposito = d.depo_codigo
                    AND s2.stoc_cantidad <= 0 )
GROUP BY d.depo_codigo, d.depo_detalle
ORDER BY 1
```

Resultado en tu base: 2 depósitos (`15` con 78 productos y `17` con 62). Si sacás el `JOIN STOCK` y dejás solo el `NOT EXISTS`, salen 22: los 2 correctos más **20 depósitos vacíos**, que cumplen "todos" sin tener nada.

Receta 6 — "Sin NULLs" / "mostrar 0 si no hay"

- Subconsulta que puede no encontrar filas → `ISNULL(( SELECT SUM(...) ... ), 0)` — **el ISNULL va AFUERA** del subselect (si no hay filas, el SUM ya da NULL y adentro no hay nada a qué aplicarle el ISNULL).
- "Todos los clientes, aunque no hayan comprado" → `FROM Cliente LEFT JOIN Factura` y `COUNT(f.fact_numero)` (no `COUNT(*)`), o directamente mandaderos con `ISNULL`. Otra salida: los que compraron `UNION ALL` los que no, con todo en 0 (ejercicio 6 b).
- `x > NULL` nunca es verdadero: si comparás contra una subconsulta, envolvela en `ISNULL`.
- 🎙️ **La trampa del LEFT JOIN** (el profe insistió con esto): en la tabla *dominada* (la que puede quedar en NULL) hasta los campos obligatorios y la PK pueden venir en NULL. Si después ponés una condición sobre ella en el `WHERE`, el `NULL = ...` da falso y el LEFT JOIN termina funcionando como un INNER ("terminó en un sinsentido"). Para "achicar la tabla dominada", **el filtro va en el ON**:

```sql
-- MAL: el WHERE sobre la dominada "mata" el LEFT JOIN (los que no compraron en 2012 desaparecen)
FROM Cliente c LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
-- BIEN: el filtro de la dominada va en el ON ("achicar la tabla dominada")
FROM Cliente c LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012
-- y en el SELECT: SUM(ISNULL(f.fact_total, 0))
```

Un filtro sobre la tabla **dominante** (ej. "que no sean consumidores finales") sí puede ir en el `WHERE`. Ejemplo de la guía (Ej. 3, "stock total de cada producto"): `Producto LEFT JOIN STOCK` + `ISNULL(SUM(stoc_cantidad), 0)`, para que también aparezcan los productos sin fila en `STOCK`.

✍️ Ejercicio resuelto 6 (a) — con LEFT JOIN

> **Enunciado:** Listar **todos** los clientes con la cantidad de facturas y el monto total que compraron en 2012. Los que no compraron en 2012 tienen que aparecer con 0 en las dos columnas.

Cómo lo pienso:

1. "Todos los clientes, aunque no hayan comprado" → `Cliente LEFT JOIN Factura`. `Cliente` es la dominante y `Factura` la dominada.
2. El filtro "2012" es sobre la dominada → va en el **ON**, no en el WHERE.

3. Contar facturas → `COUNT(f.fact_numero)`, no `COUNT(*)`: el cliente sin compras tiene una fila con todo NULL del lado de `Factura`. `COUNT(*)` la cuenta (daría 1) y `COUNT(columna)` ignora el NULL (da 0).
4. Monto → `ISNULL(SUM(...), 0)`: el SUM de puros NULL da NULL.

```
SELECT c.clie_codigo, c.clie_razon_social,
    COUNT(f.fact_numero)      AS facturas_2012,
    ISNULL(SUM(f.fact_total), 0) AS monto_2012
FROM Cliente c
LEFT JOIN Factura f ON f.fact_cliente = c.clie_codigo
            AND YEAR(f.fact_fecha) = 2012          -- filtro de la dominada: en el ON
GROUP BY c.clie_codigo, c.clie_razon_social
ORDER BY 4 DESC, 1
```

Resultado en tu base: 3687 filas, una por cliente (arriba `01742` METHANET con 190 facturas y 166477.34). Con `YEAR(f.fact_fecha) = 2012` en el **WHERE** salen solo 66: el LEFT JOIN se convirtió en un INNER y desaparecieron todos los que no compraron.

✍️ Ejercicio resuelto 6 (b) — la misma idea, partida en dos con UNION ALL

🎙️ En la clase de operadores de conjunto el profe resolvió el Ej. 14 de la guía **sin LEFT JOIN**. La idea: separar a los clientes en dos grupos que no se pisan, resolver cada uno por su lado y apilarlos.

> **Enunciado (Ej. 14 de la guía):** Una estadística de ventas por cliente con: código del cliente, cantidad de veces que compró en el último año, promedio por compra en el último año, cantidad de productos diferentes que compró en el último año y monto de la mayor compra del último año. Se deben devolver **todos** los clientes, ordenados por la cantidad de veces que compraron en el último año. No puede haber NULLs en ninguna columna.

Cómo lo pienso:

1. "Último año" admite varias lecturas (el último año cargado, el año en curso o los últimos 12 meses). 🎙️ El profe lo simplificó a **2012**, el último año de la base. En el parcial, **escribí el supuesto**.
2. **Parte de arriba, los que compraron:** `Cliente JOIN Factura` con `YEAR = 2012`, agrupado por cliente. Cantidad = `COUNT(*)`, promedio = `AVG(fact_total)`, mayor compra = `MAX(fact_total)`. Como es un INNER JOIN, todo cliente que aparece tiene al menos una factura: no hay NULLs.
3. "Productos diferentes" vive en `Item_Factura`: si la sumo a la mesa, multiplica las filas y rompe el COUNT y el AVG (6.3). Va como **mandadero** con `COUNT(DISTINCT item_producto)`.
4. **Parte de abajo, los que no compraron:** `NOT EXISTS` de facturas en 2012, con todas las columnas en `0`.
5. **UNION ALL, no UNION:** un cliente está en una parte o en la otra, nunca en las dos. No hay repetidos que buscar.
6. El ORDER BY va una sola vez, al final, y ordena todo.

```sql
SELECT c.clie_codigo,
       COUNT(*)        AS compras,
       AVG(f.fact_total) AS promedio,
       (SELECT COUNT(DISTINCT i2.item_producto)          -- mandadero: Item_Factura multiplicaría
          FROM Factura f2
          JOIN Item_Factura i2 ON i2.item_tipo = f2.fact_tipo AND i2.item_sucursal = f2.fact_sucursal
                              AND i2.item_numero = f2.fact_numero
         WHERE f2.fact_cliente = c.clie_codigo
           AND YEAR(f2.fact_fecha) = 2012) AS productos_distintos,
       MAX(f.fact_total) AS mayor_compra
FROM Cliente c
JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY c.clie_codigo
UNION ALL                                            -- los que no compraron, todo en 0
SELECT c.clie_codigo, 0, 0, 0, 0
FROM Cliente c
WHERE NOT EXISTS (SELECT 1 FROM Factura f2
                  WHERE f2.fact_cliente = c.clie_codigo AND YEAR(f2.fact_fecha) = 2012)
ORDER BY 2 DESC
```

Resultado en tu base: 3687 filas, una por cliente y ningún NULL. Arriba aparece `01742` con 190 compras, promedio 876.20, 121 productos distintos y una compra máxima de 4421.16. Las 66 primeras son los que compraron y el resto tiene todo en 0. Es el mismo resultado que el ejercicio 6 (a) con LEFT JOIN; elegí la forma que te resulte más clara.

Receta 7 — "% sobre el total del año"

```sql
SUM(i.item_cantidad * i.item_precio) * 100.0
  / ( SELECT SUM(i2.item_cantidad * i2.item_precio) FROM ... WHERE YEAR(f2.fact_fecha) = YEAR(f.fact_fecha) )
```

(El `100.0` evita la división entera.)

✍️ Ejercicio resuelto 7

> **Enunciado:** Para 2012, mostrar cada familia de productos con lo facturado (cantidad × precio) y el porcentaje que representa sobre el total facturado en el año. Ordenar de mayor a menor porcentaje.

Cómo lo pienso:

1. *Sujeto:* año + familia → agrupo renglones por `YEAR(fecha)` y familia.
2. *Facturado:* sale de la mesa → `SUM(item_cantidad * item_precio)` (por producto no se usa `fact_total`, ver 8.1).
3. *Total del año:* es **otro filtro** (todas las familias, no solo la de esta fila) → mandadero correlacionado por año.
4. Agrupo por `fami_id` y no solo por el detalle, porque puede haber dos familias con el mismo nombre (en tu base hay dos "KRAFT").

```sql
SELECT YEAR(f.fact_fecha) AS anio, fa.fami_detalle,
    SUM(i.item_cantidad * i.item_precio) AS facturado,
    SUM(i.item_cantidad * i.item_precio) * 100.0
     / ( SELECT SUM(i2.item_cantidad * i2.item_precio)        -- total del año (todas las familias)
         FROM Item_Factura i2
         JOIN Factura f2 ON f2.fact_tipo = i2.item_tipo AND f2.fact_sucursal = i2.item_sucursal AND f2.fact_numero =
i2.item_numero
         WHERE YEAR(f2.fact_fecha) = YEAR(f.fact_fecha) ) AS porcentaje_del_anio
FROM Item_Factura i
JOIN Factura f ON f.fact_tipo = i.item_tipo AND f.fact_sucursal = i.item_sucursal AND f.fact_numero = i.item_numero
JOIN Producto p ON p.prod_codigo = i.item_producto
JOIN Familia fa ON fa.fami_id = p.prod_familia
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY YEAR(f.fact_fecha), fa.fami_id, fa.fami_detalle
ORDER BY 1, 4 DESC
```

Resultado en tu base: primero PORTAFOLIO PRIMARIO con 21.33%, después SIN ASIGNACION con 21.14%. Como el mandadero usa `YEAR(f.fact_fecha)` de la fila y no un 2012 fijo, si sacás el `WHERE` la misma consulta calcula el porcentaje de cada año por separado.

Receta 8 — "Ordenar primero los que..., luego el resto"

```sql
ORDER BY CASE WHEN (condición, típicamente EXISTS(...)) THEN 0 ELSE 1 END, otro_criterio
```

✍️ Ejercicio resuelto 8

> **Enunciado:** Listar los empleados (código, apellido, nombre) mostrando primero los que son encargados de algún depósito y después el resto. Dentro de cada grupo, ordenar por apellido. Agregar una columna que diga `Encargado` o `-`.

Cómo lo pienso: la condición "es encargado" es un `EXISTS` sobre `DEPOSITO`. Si la convierto en 0/1 con un `CASE`, ordenar por ese número pone primero los 0. La misma condición sirve para armar la columna de texto.

```sql
SELECT e.empl_codigo, e.empl_apellido, e.empl_nombre,
    CASE WHEN EXISTS ( SELECT 1 FROM DEPOSITO d WHERE d.depo_encargado = e.empl_codigo )
        THEN 'Encargado' ELSE '-' END AS rol
FROM Empleado e
ORDER BY CASE WHEN EXISTS ( SELECT 1 FROM DEPOSITO d WHERE d.depo_encargado = e.empl_codigo )
         THEN 0 ELSE 1 END,                    -- 1er criterio: encargados arriba
    e.empl_apellido                            -- 2do criterio: apellido
```

Resultado en tu base: primero los 5 encargados (Borr, Gonzalez, Perez, Quiroga, Trap) y después los otros 4 (Doind, Jand, Trap, Trisoli). Fijate que hay dos "Trap" y cada uno queda en su grupo.

¿Por qué no `ORDER BY 4`? Porque ordenaría alfabéticamente el texto (`-` antes que `Encargado`) y quedarían al revés. Funciona solo si elegís bien los textos. El `CASE` con 0/1 no depende de eso.

Receta 9 — "No se permite subconsulta en el FROM ni CTE"

No te asustes: con las herramientas de arriba nunca la necesitás. Reemplazos:

- "filtrar por un total" → `HAVING`.
- "filtrar por un ranking" → `IN (SELECT TOP N ...)`.
- "comparar totales de dos períodos" → CASE o año ancla.
- "dato de otro nivel" → mandadero en el SELECT.

✍️ Ejercicio resuelto 9

> **Enunciado:** Mostrar los clientes cuyo monto comprado en 2012 supera el promedio de compra por cliente de ese año. **No se permiten subconsultas en el FROM ni CTE.**

Cómo lo pienso: la tentación es calcular "el monto de cada cliente" en una tabla temporal y después hacer el `AVG` encima:

```
-- PROHIBIDO en este enunciado (subconsulta en el FROM):
SELECT AVG(x.monto)
FROM ( SELECT fact_cliente, SUM(fact_total) AS monto
       FROM Factura WHERE YEAR(fact_fecha) = 2012
       GROUP BY fact_cliente ) x
```

Pero "promedio de monto por cliente" es lo mismo que **total facturado ÷ cantidad de clientes que compraron**, y eso sale con una sola consulta sin FROM anidado. La condición es sobre el total de cada cliente → `HAVING`.

```
SELECT c.clie_codigo, c.clie_razon_social, SUM(f.fact_total) AS monto_2012
FROM Cliente c
JOIN Factura f ON f.fact_cliente = c.clie_codigo
WHERE YEAR(f.fact_fecha) = 2012
GROUP BY c.clie_codigo, c.clie_razon_social
HAVING SUM(f.fact_total) > ( SELECT SUM(f2.fact_total) / COUNT(DISTINCT f2.fact_cliente)   -- promedio por cliente
                             FROM Factura f2
                             WHERE YEAR(f2.fact_fecha) = 2012 )
ORDER BY 3 DESC
```

Resultado en tu base: las dos formas dan el mismo promedio (11111.26) y la consulta devuelve 14 clientes, de `01742` (166477.34) a `01745` (11885.73).

Receta 10 — 🎙️ Reglas de subconsultas que marcó el profe

Qué tiene que devolver la subconsulta según dónde la ponés:

Dónde	Qué puede devolver	Nota
`WHERE x IN (...)` / `NOT IN`	**1 columna**, 0..N filas	con 2 columnas da error y "no tiene sentido"
`WHERE x = (...)` , `>` , `<`	**1 valor**	si puede traer varias filas, usá `IN`
`SELECT (...) AS col` (mandadero)	**1 fila, 1 columna**	un agregado **sin GROUP BY** lo garantiza siempre (puede dar 0 o NULL, pero da una fila)
`UPDATE ... SET col = (...)`	**1 fila, 1 columna**	igual que el anterior
`FROM (...) alias`	filas × columnas	**alias obligatorio**; es la más cara: va entera a memoria
`EXISTS (...)`	verdadero/falso	no lleva nada a memoria; `SELECT 1` o `SELECT *` da igual

La regla de fondo (clase 1): un agregado **sin GROUP BY** sobre un conjunto vacío **devuelve igual una fila** (`COUNT` = 0; `SUM` , `MIN` , `MAX` y `AVG` = NULL). **Con GROUP BY**, si no quedan filas no se arma ningún grupo y se devuelven **0 filas**. Además, `AVG` y `COUNT(col)` ignoran los NULL.

La trampa del EXISTS con agregado: por esa regla, `EXISTS (SELECT COUNT(*) ... )` o `EXISTS (SELECT SUM(...) ...)` **sin GROUP BY** es **siempre verdadero**, porque el agregado devuelve una fila aunque no haya datos (devuelve 0 o NULL). El profe lo llamó "mal programado". Si necesitás una condición sobre un total, usá `HAVING` adentro del EXISTS (como en el año ancla, que tiene `GROUP BY`) o compará el mandadero afuera: `WHERE (SELECT COUNT(*) ...) >= 5` .

✍️ Ejercicio resuelto 10

> **Enunciado:** Mostrar los clientes que tienen al menos 100 facturas en 2012.

El error típico (y por qué está mal):

```
-- MAL: devuelve TODOS los clientes
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
WHERE EXISTS ( SELECT COUNT(*) FROM Factura f
        WHERE f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012 )
```

`COUNT(*)` sin `GROUP BY` **siempre devuelve una fila** (aunque valga 0), así que el `EXISTS` siempre encuentra algo. En tu base devuelve los 3687 clientes, incluso los que no compraron nunca. Y además el "100" no aparece en ningún lado.

Bien, opción 1 — mandadero comparado afuera (la más clara):

```
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
WHERE ( SELECT COUNT(*) FROM Factura f
    WHERE f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012 ) >= 100
ORDER BY 1
```

Bien, opción 2 — EXISTS con HAVING: el `HAVING` sí puede "matar" la única fila del agregado, y entonces el `EXISTS` da falso.

```sql
SELECT c.clie_codigo, c.clie_razon_social
FROM Cliente c
WHERE EXISTS ( SELECT 1 FROM Factura f
        WHERE f.fact_cliente = c.clie_codigo AND YEAR(f.fact_fecha) = 2012
        HAVING COUNT(*) >= 100 )
ORDER BY 1
```

Resultado en tu base: las dos opciones correctas devuelven los mismos 9 clientes (`01621` , `01742` , `01743` , `01745` , `01747` , `01751` , `01772` , `02088` , `02183`).

Correlación = una ejecución por fila. La subconsulta correlacionada se ejecuta una vez por cada fila (o valor distinto) de afuera, y el resultado se le asigna a *esa* fila. Por eso mantené chico lo de afuera (filtrá antes) y no metas subconsultas grandes sobre las tablas **transaccionales** (`Factura` , `Item_Factura`): ahí está el volumen.

Criterios del profe en los ejercicios de la guía:

- Para "mostrar código y detalle sin repetir", **agrupá en vez de usar** `DISTINCT` : dijo que el DISTINCT "no es lo recomendable" y que con el GROUP BY después podés agregar los SUM.
- En el `GROUP BY` agregá también el detalle que querés mostrar: no cambia la cantidad de grupos (el detalle depende de la PK).
- `ORDER BY 2` (número de columna) solo funciona si esa columna **se muestra**. Si ordenás por algo que no está en el SELECT, escribí la expresión. Ordenar por **alias** funciona, pero el profe dijo que "a su gusto no es recomendable", porque se presta a confusión si el alias coincide con el nombre de una columna. En orden ascendente, los NULL quedan **primero**.
- `= NULL` y `<> NULL` dan falso siempre: se pregunta con `IS NULL` / `IS NOT NULL` . Un `CASE` sin `ELSE` devuelve NULL cuando no entra en ninguna rama, y todas las ramas tienen que devolver el mismo tipo de dato.
- Ej. 13, "compuestos por más de 2 productos": lo resolvió con `HAVING SUM(comp_cantidad) > 2` , o sea contando unidades (la linterna con 2 pilas = 1 + 2 = 3).
- Ej. 7, "% de diferencia respecto del menor": `(MAX - MIN) * 100 / MIN` ; "que posean stock" = `stoc_cantidad > 0` (el modelo admite filas con 0 o con negativos).

Receta 11 — "Los N más… y también los N menos…" (dos listas en un solo resultado)

Cuando el enunciado pide **dos listas distintas en el mismo resultado**, armá cada una con su propio SELECT y apilalas con `UNION ALL` (7.8). Las dos partes tienen que devolver las mismas columnas; una columna fija con un texto ('Más vendido' / 'Menos vendido') le dice al lector de qué lista viene cada fila.

- Cada ranking va adentro de un `IN (SELECT TOP N ...)` , no como TOP de la parte: `SELECT TOP 10 ... ORDER BY ... UNION ...` no compila, porque en un operador de conjunto hay un solo ORDER BY, al final.
- Si cada fila necesita un dato "el que más" (el cliente que más lo compró), es un mandadero TOP 1 (receta 1), repetido en las dos partes.
- ⚠ **Empates en el borde.** Si hay más candidatos empatados que lugares, el TOP elige cualquiera y el resultado puede cambiar de una corrida a otra. Agregá un desempate al ORDER BY (por ejemplo, el código) y escribí el criterio como supuesto.

✍️ Ejercicio resuelto 11

> **Enunciado (Ej. 10 de la guía):** Mostrar los 10 productos más vendidos en la historia y también los 10 productos menos vendidos en la historia. Además, para esos productos, mostrar quién fue el cliente que más compras realizó.

Cómo lo pienso:

1. Dos listas en un resultado → `UNION ALL` de dos SELECT con el mismo sujeto, `FROM Producto`.
2. "Los 10 más vendidos" → `prod_codigo IN (SELECT TOP 10 item_producto ... GROUP BY item_producto ORDER BY SUM(item_cantidad) DESC)`. "Los 10 menos" es lo mismo con `ASC`. 🎙️ El profe aclaró que "menos vendido" significa que **tuvo ventas**: partiendo de `Item_Factura` ya quedan afuera los que nunca se vendieron.
3. ⚠️ En tu base hay **24 productos empatados con 1 sola unidad vendida**. Sin desempate, "los 10 menos vendidos" son 10 cualesquiera de esos 24. Desempato por código.
4. "El cliente que más lo compró" → mandadero TOP 1 agrupado por cliente (receta 1).
5. Una columna fija dice de qué lista viene cada fila. UNION ALL porque un producto no puede estar entre los 10 más y los 10 menos a la vez (se vendieron 661 productos distintos).

```
SELECT p.prod_codigo, p.prod_detalle, 'Más vendido' AS ranking,
    (SELECT TOP 1 f2.fact_cliente
      FROM Item_Factura i2
      JOIN Factura f2 ON f2.fact_tipo = i2.item_tipo AND f2.fact_sucursal = i2.item_sucursal
                AND f2.fact_numero = i2.item_numero
      WHERE i2.item_producto = p.prod_codigo
      GROUP BY f2.fact_cliente
      ORDER BY SUM(i2.item_cantidad) DESC) AS cliente_que_mas_compro
FROM Producto p
WHERE p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                GROUP BY item_producto
                ORDER BY SUM(item_cantidad) DESC, item_producto)
UNION ALL
SELECT p.prod_codigo, p.prod_detalle, 'Menos vendido',
    (SELECT TOP 1 f2.fact_cliente
      FROM Item_Factura i2
      JOIN Factura f2 ON f2.fact_tipo = i2.item_tipo AND f2.fact_sucursal = i2.item_sucursal
                AND f2.fact_numero = i2.item_numero
      WHERE i2.item_producto = p.prod_codigo
      GROUP BY f2.fact_cliente
      ORDER BY SUM(i2.item_cantidad) DESC)
FROM Producto p
WHERE p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                GROUP BY item_producto
                ORDER BY SUM(item_cantidad) ASC, item_producto)   -- desempate por código
ORDER BY ranking, prod_detalle
```

Resultado en tu base: 20 filas. Entre los más vendidos están las dos linternas, MARLBORO BOX y los TANG, y el cliente `01742` es el que más compró la mitad de ellos (🎙️ el profe probó la linterna `00001718` y le dio el mismo cliente). Los 10 menos vendidos son los 10 primeros códigos de los 24 empatados en 1 unidad.

Sin operador de conjunto también se puede, con un OR y un CASE para la etiqueta (🎙️ el profe mostró esta variante):

```
SELECT p.prod_codigo, p.prod_detalle,
    CASE WHEN p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                    GROUP BY item_producto ORDER BY SUM(item_cantidad) DESC, item_producto)
        THEN 'Más vendido' ELSE 'Menos vendido' END AS ranking
FROM Producto p
WHERE p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                GROUP BY item_producto ORDER BY SUM(item_cantidad) DESC, item_producto)
  OR p.prod_codigo IN (SELECT TOP 10 item_producto FROM Item_Factura
                GROUP BY item_producto ORDER BY SUM(item_cantidad) ASC, item_producto)
ORDER BY 3, 2
```

Se repite el ranking de los más vendidos en el CASE y en el WHERE. Por eso, cuando hay dos listas, la UNION ALL se lee mejor.

Para llevarte

- Sujeto en el FROM; cada columna, ¿mesa o mandadero?
- Dos tablas hijas + SUM = mandadero.
- Años con nombre: CASE. Años genéricos: año ancla.
- "Solo / todos": NOT EXISTS del contraejemplo (y cuidado con el vacío).
- Dos listas en un resultado: UNION ALL de dos SELECT, con el TOP adentro de un IN.

Capítulo 9 — Programar en la base: funciones y cursores

Hasta acá cada consulta era una sola instrucción. Desde este capítulo vas a escribir **programas guardados dentro de la base**: con variables, condiciones y ciclos. 🎙️ El profe lo presentó así: la lógica de negocio se programa dentro de la base de datos, *"ya no en un backend"*.

9.1 Los tres objetos programables

Objeto	Cómo se ejecuta	Qué puede hacer
Función	se la invoca dentro de una consulta	**solo leer** y devolver un valor o una tabla
Procedimiento (stored procedure, SP)	se lo invoca con `EXEC`	leer, **modificar datos**, manejar transacciones, crear tablas
Trigger	**se dispara solo** ante un INSERT, UPDATE o DELETE	lo mismo que un SP, sin que nadie lo llame

🎙️ Lo que diferencia a la función: *"no abre transacciones, no cierra transacciones, no modifica parámetros del sistema, no modifica datos"*. Lo que comparten los tres: se declaran igual las variables y los parámetros, y se usan los mismos bloques IF y WHILE. *"Lo que vemos hoy en funciones se repite para procedures."*

9.2 La gramática común

```
DECLARE @total DECIMAL(12,2) = 0, @cant INT, @porc DECIMAL(5,2) = 0.57   -- varias en una línea
SET @cant = 10                                  -- asignar un valor
SELECT @total = SUM(fact_total) FROM Factura WHERE YEAR(fact_fecha) = 2012 -- asignar desde una consulta

IF @total > 1000
BEGIN
   SET @cant = @cant + 1
END
ELSE
   SET @cant = 0          -- una sola instrucción: BEGIN/END opcional

WHILE @cant > 0
BEGIN
   SET @cant = @cant - 1
   IF @cant = 5 BREAK       -- BREAK sale del WHILE (no de la función)
END
```

🎙️ Reglas que remarcó el profe:

- Las variables y los parámetros propios empiezan con `@`. Las variables **del sistema**, con `@@` (como `@@FETCH_STATUS` y `@@ROWCOUNT`, que vas a ver enseguida).
- Los decimales se escriben con **punto** (`0.57`), porque la coma separa términos.
- Una variable que no inicializás vale **NULL**. Y `NULL > 0` no es verdadero, así que un `WHILE @x > 0` con `@x` en NULL nunca entra.
- ⚠️ **`SELECT @v = col FROM …`** tiene dos comportamientos que no dan error:

    - Si la consulta **no devuelve filas**, `@v` **conserva el valor que tenía** (NULL si no la inicializaste).
    - Si devuelve **varias filas**, `@v` se queda con el valor de **una sola** de ellas, sin avisar.
    - Por eso conviene inicializar las variables y asegurarse de que la consulta devuelva una fila (por clave primaria o con un agregado).
- `GO` separa lotes. `CREATE FUNCTION`, `CREATE PROCEDURE`, `CREATE VIEW` y `CREATE TRIGGER` tienen que ser la primera instrucción de su lote, así que antes de cada uno va un `GO`.

9.3 Funciones escalares

Una función escalar recibe parámetros y devuelve **un valor**:

```
CREATE FUNCTION dbo.fn_estado_deposito (@producto CHAR(8), @deposito CHAR(2))
RETURNS VARCHAR(100)
AS
BEGIN
  DECLARE @cant DECIMAL(12,2), @max DECIMAL(12,2), @estado VARCHAR(100)

  SELECT @cant = stoc_cantidad, @max = stoc_stock_maximo
  FROM STOCK
  WHERE stoc_producto = @producto AND stoc_deposito = @deposito

  IF @cant < @max
    SET @estado = 'OCUPACION DEL DEPOSITO '
            + CAST(CAST(@cant * 100.0 / @max AS DECIMAL(5,2)) AS VARCHAR) + ' %'
  ELSE
    SET @estado = 'DEPOSITO COMPLETO'

  RETURN @estado
END
GO
SELECT dbo.fn_estado_deposito('00000102', '00')
```

Este es el ejercicio T1 de la guía. 🎙️ En la puesta en común, el profe sugirió calcular el porcentaje y, *"antes del RETURN, tirás un IF"* para armar el texto.

Reglas de las funciones:

- 🎙️ **La última instrucción tiene que ser un RETURN.** Si hay algún camino del código que termina sin devolver nada, no compila. El profe lo mostró comentando el RETURN final: el error dice *"la última instrucción de una función debe ser el RETURN"*.
- Se invocan **con el esquema**: `dbo.fn_estado_deposito(...)`.
- Una función que no encuentra nada puede devolver NULL, igual que `YEAR(NULL)`.
- 🎙️ Si el resultado sale de una sola consulta, se puede escribir directo `RETURN (SELECT ...)`, sin variable.

Dónde se pueden usar: en todos los lugares donde va un valor: SELECT, WHERE, JOIN ... ON, GROUP BY, HAVING, ORDER BY, el SET de un UPDATE y los VALUES de un INSERT.

🎙️ **Performance.** Una función en el WHERE se ejecuta **una vez por fila** de la tabla: 100.000 clientes son 100.000 llamadas, y para el motor la función es una caja negra que no puede optimizar. Por eso el profe recomendó **pasarle a la función los datos que ya leíste** (por ejemplo, apellido y nombre) en lugar de la clave para que vuelva a buscarlos con otro SELECT. Además, así la función sirve para más casos: le da formato a un cliente, a un empleado o a un proveedor.

9.4 Funciones de tabla

Una función también puede devolver **un conjunto de filas**, y entonces se usa en el FROM, como una tabla y con alias:

```
CREATE FUNCTION dbo.fn_ventas_por_cliente (@anio INT)
RETURNS TABLE
AS
RETURN ( SELECT fact_cliente, COUNT(*) AS facturas, SUM(fact_total) AS total
        FROM Factura
        WHERE YEAR(fact_fecha) = @anio
        GROUP BY fact_cliente )
GO
SELECT c.clie_razon_social, v.total
FROM dbo.fn_ventas_por_cliente(2012) v
JOIN Cliente c ON c.clie_codigo = v.fact_cliente
ORDER BY v.total DESC
```

Es como una vista que recibe parámetros.

9.5 Cursores: recorrer fila por fila

Todo lo que vimos hasta acá trabaja con **conjuntos** de filas de una vez. A veces necesitás procesar **una fila por vez**, y para eso está el cursor. 🎙️ El profe lo definió como *"un SELECT que se ejecuta, el resultado va a memoria y se recorre con un loop"*. Tiene cinco pasos:

```
DECLARE @clie CHAR(6), @razon CHAR(100)

DECLARE c_clientes CURSOR FOR                    -- 1) declarar: el SELECT todavía NO se ejecuta
   SELECT clie_codigo, clie_razon_social
   FROM Cliente
   WHERE clie_codigo BETWEEN '00000' AND '00003'

OPEN c_clientes                                  -- 2) abrir: ACÁ se ejecuta el SELECT
FETCH NEXT FROM c_clientes INTO @clie, @razon    -- 3) traer la primera fila (una variable por columna)

WHILE @@FETCH_STATUS = 0                         --   0 = trajo una fila; -1 = no hay más
BEGIN
   PRINT @clie + ' - ' + RTRIM(@razon)
   FETCH NEXT FROM c_clientes INTO @clie, @razon --   traer la siguiente (sin esto: loop infinito)
END

CLOSE c_clientes                                 -- 4) cerrar
DEALLOCATE c_clientes                            -- 5) liberar la memoria
```

- En el `INTO` va **una variable por cada columna** del SELECT, en el mismo orden.
- Si el SELECT no devuelve filas, nunca entra al WHILE.
- 🎙️ `DEALLOCATE` *"es exactamente el free de C"*. Si te lo olvidás, el cursor ocupa memoria hasta que se cierra la sesión.
- Para modificar la fila donde está parado el cursor: se declara `FOR UPDATE OF columna` y se usa `UPDATE ... WHERE CURRENT OF c_clientes`.
- 🎙️ **¿Cursor o vista?** Los dos tienen un SELECT. Pero la vista se usa desde una consulta. El cursor existe solo dentro de un objeto programado, para *"leer novedades y, a medida que voy leyendo, actualizo"*.

Regla práctica: si el problema se resuelve con una consulta de conjuntos (un UPDATE con JOIN, un INSERT … SELECT), es preferible al cursor. El cursor tiene sentido cuando cada fila depende de lo que pasó con la anterior, o para recorrer una jerarquía, que es lo que sigue.

9.6 Funciones recursivas

Cuando el enunciado dice *"directa o indirectamente"*, *"a cualquier nivel"* o *"no se conocen los niveles de anidamiento"*, está pidiendo **recursión**: una función que se llama a sí misma. Siempre tiene la misma forma: un **caso base**, que se resuelve sin llamarse, y un **caso recursivo**, que se llama para los hijos.

T15. Precio de un producto compuesto. Si es simple, su precio. Si es compuesto, la suma de (cantidad × precio de cada componente), y cada componente puede ser compuesto a su vez:

```
CREATE FUNCTION dbo.fn_precio_real (@prod CHAR(8))
RETURNS DECIMAL(12,2)
AS
BEGIN
   IF NOT EXISTS ( SELECT 1 FROM Composicion WHERE comp_producto = @prod )     -- caso base: es simple
      RETURN ( SELECT prod_precio FROM Producto WHERE prod_codigo = @prod );
   RETURN ( SELECT SUM(c.comp_cantidad * dbo.fn_precio_real(c.comp_componente))  -- caso recursivo
         FROM Composicion c
         WHERE c.comp_producto = @prod );
END
GO
SELECT prod_codigo, prod_detalle FROM Producto WHERE dbo.fn_precio_real(prod_codigo) > 100
```

T11. Cantidad de empleados a cargo, directa o indirectamente. 🎙️ El profe lo resolvió con un cursor: recorre los subordinados directos y, por cada uno, suma 1 (el directo) más lo que devuelve la función para ese subordinado (sus indirectos). También comentó que una solución que mira un solo nivel está mal: hay que recursionar.

```
CREATE FUNCTION dbo.fn_empleados_a_cargo (@jefe NUMERIC(6))
RETURNS INT
AS
BEGIN
   DECLARE @total INT = 0, @emp NUMERIC(6)
   DECLARE c_sub CURSOR LOCAL FOR
      SELECT empl_codigo FROM Empleado WHERE empl_jefe = @jefe
   OPEN c_sub
   FETCH NEXT FROM c_sub INTO @emp
   WHILE @@FETCH_STATUS = 0
   BEGIN
      SET @total = @total + 1 + dbo.fn_empleados_a_cargo(@emp)   -- el directo + sus indirectos
      FETCH NEXT FROM c_sub INTO @emp
   END
   CLOSE c_sub
   DEALLOCATE c_sub
   RETURN @total
END
```

(`LOCAL` hace que cada nivel de la recursión tenga su propio cursor y no choquen los nombres. El enunciado completo de T11 agrega un filtro sobre los códigos, que se suma en el WHERE del cursor.)

La misma función, sin cursor, en el estilo de T15:

```
RETURN ( SELECT COUNT(*) + ISNULL(SUM(dbo.fn_empleados_a_cargo(empl_codigo)), 0)
      FROM Empleado WHERE empl_jefe = @jefe )
```

🎙️ Desventaja de la recursión: cada llamada queda abierta hasta que terminan las de abajo. Con cursores es peor, porque cada nivel tiene su cursor en memoria. (SQL Server corta a los 32 niveles de anidamiento.)

Para llevarte

- Función = solo lee y devuelve. Termina en RETURN en todos los caminos. Se invoca con `dbo.`.
- `SELECT @v = ...` sin filas deja el valor anterior: inicializá.
- Cursor: DECLARE, OPEN, FETCH, WHILE @@FETCH_STATUS = 0 (con otro FETCH al final), CLOSE, DEALLOCATE.
- "A cualquier nivel" = recursión: caso base + caso recursivo.

Capítulo 10 — Procedimientos y transacciones

10.1 Stored procedures

Un procedimiento es un programa que se ejecuta a pedido y **puede modificar datos**. Su forma:

```
CREATE PROCEDURE dbo.sp_nombre (@param1 CHAR(8), @monto DECIMAL(12,2), @resultado INT OUTPUT)
AS
BEGIN
   SET NOCOUNT ON;        -- no mostrar "n filas afectadas" en cada instrucción
   ...
END
GO
-- invocación: OUTPUT va en la definición y también en la llamada
DECLARE @r INT
EXEC dbo.sp_nombre '00001705', 10, @r OUTPUT
SELECT @r
```

🎙️ Lo que remarcó el profe:

- No está obligado a devolver nada, pero *"sería recomendable que devuelva algo"*: un parámetro **OUTPUT** con un código de resultado (0 = OK, otro número = un error). *"En un sistema real saldría de una tabla de errores."*
- El parámetro OUTPUT **no es el RETURN**: lo podés ir cambiando durante el proceso sin cortar la ejecución. El `RETURN`, en cambio, termina el procedimiento en esa línea.
- Un SP puede **crear tablas** (una función no puede). Por eso los ejercicios T5 y T7 de la guía (crear y llenar una tabla) son procedimientos.

Un ejemplo de la guía (T3): la tabla Empleado tiene que tener un único empleado sin jefe (el gerente general). Si hay varios, el gerente es el de mayor salario (con desempate por antigüedad), y los demás pasan a depender de él. Hay que devolver cuántos empleados sin jefe había al principio.

```
CREATE PROCEDURE dbo.sp_unico_gerente (@sin_jefe_al_inicio INT OUTPUT)
AS
BEGIN
   SET NOCOUNT ON;
   DECLARE @gerente NUMERIC(6)

   SELECT @sin_jefe_al_inicio = COUNT(*) FROM Empleado WHERE empl_jefe IS NULL

   SELECT TOP 1 @gerente = empl_codigo
   FROM Empleado
   WHERE empl_jefe IS NULL
   ORDER BY empl_salario DESC, empl_ingreso ASC        -- mayor salario; desempate: más antiguo

   UPDATE Empleado SET empl_jefe = @gerente
   WHERE empl_jefe IS NULL AND empl_codigo <> @gerente
END
```

Para "alta o modificación" en una sola instrucción, el profe usó **MERGE** (si no existe, inserta; si existe, actualiza):

```
MERGE Cliente AS destino
USING ( SELECT @codigo AS codigo, @razon AS razon ) AS origen
  ON destino.clie_codigo = origen.codigo
WHEN MATCHED    THEN UPDATE SET clie_razon_social = origen.razon
WHEN NOT MATCHED THEN INSERT (clie_codigo, clie_razon_social) VALUES (origen.codigo, origen.razon);
```

10.2 Transacciones explícitas

En el capítulo 5 viste que cada instrucción es atómica por sí sola. Con un procedimiento podés **agrupar varias instrucciones** en una transacción:

```
BEGIN TRANSACTION
  UPDATE ...     -- si algo sale mal...
  ROLLBACK       -- ...se deshace todo lo hecho desde el BEGIN
  ...
COMMIT           -- si todo salió bien, se confirma
```

🎙️ El profe lo comparó con los paréntesis, que se abren una vez y se cierran una vez. Con las transacciones no es así: `BEGIN TRANSACTION` y el `COMMIT` final se escriben una vez, pero puede haber **varios ROLLBACK**, uno por cada camino de error. *"En tiempo de ejecución se va uno solo."* Un procedimiento también puede tener varias transacciones seguidas.

⚠️ Las tres reglas que más se equivocan:

1. **ROLLBACK no corta el procedimiento.** 🎙️ *"No sigue la transacción, pero sigue el stored procedure."* Después de cada ROLLBACK hay que cargar el código de error y hacer `RETURN`.
2. **ROLLBACK no deshace las variables.** *"Di vuelta atrás la transacción, no los valores de las variables."*
3. **Lo que ya se confirmó con COMMIT no se deshace** con un ROLLBACK posterior. 🎙️ En el ejemplo de la transferencia, el profe usó esto a propósito: primero registra *"intento de transferencia"* y lo confirma. Así, aunque después la transferencia falle, queda el registro.

10.3 Variables del sistema útiles

- `@@ROWCOUNT`: cuántas filas afectó la **última** instrucción. ⚠️ Un UPDATE o un DELETE que no encuentra filas **no da error**. 🎙️ Para saber si existía lo que querías modificar, preguntá `IF @@ROWCOUNT = 0` justo después del UPDATE, *"para no hacer dos SELECT"* (uno para ver si existe y otro para actualizar).
- `@@IDENTITY`: el último valor autonumérico generado (por ejemplo, el ID que acabás de insertar).
- `@@TRANCOUNT`: cuántas transacciones hay abiertas.
- `@@FETCH_STATUS`: el estado del último FETCH de un cursor.

10.4 Manejo de errores: código defensivo y TRY/CATCH

🎙️ El orden que propuso el profe:

1. **Primero, el código defensivo**, antes de abrir ninguna transacción: parámetros nulos, origen igual a destino, montos menores o iguales a cero. Si algo falla, cargás el código de error y `RETURN`.

2. **Después, la transacción, dentro de un `TRY`.** Si falla una restricción o un trigger, la ejecución salta al `CATCH`.
3. **En el CATCH** se distinguen los errores *"todavía esperables"* (por ejemplo, la cuenta no tenía saldo) del *"error puro del DBMS"*, que se informa con `ERROR_NUMBER()` y `ERROR_MESSAGE()`.

La plantilla completa:

```
CREATE PROCEDURE dbo.sp_nombre (@param1 CHAR(8), @monto DECIMAL(12,2), @resultado INT OUTPUT)
AS
BEGIN
  SET NOCOUNT ON;
  -- 1) código defensivo: validar parámetros ANTES de abrir la transacción
  IF @param1 IS NULL OR @monto <= 0
  BEGIN
    SET @resultado = 1;        -- código de error
    RETURN;                    -- RETURN corta el procedimiento
  END
  BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE ... WHERE clave = @param1;
    IF @@ROWCOUNT = 0          -- no existía: en vez de hacer un SELECT antes
    BEGIN
      ROLLBACK TRANSACTION;    -- deshace la transacción, pero el SP SIGUE corriendo
      SET @resultado = 2;
      RETURN;                  -- por eso, después de cada ROLLBACK, RETURN
    END
    ...
    COMMIT TRANSACTION;
    SET @resultado = 0;        -- 0 = OK
  END TRY
  BEGIN CATCH                  -- falló una constraint o un trigger
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    SET @resultado = 99;
    PRINT ERROR_MESSAGE();     -- ERROR_NUMBER(), ERROR_MESSAGE(): el error "puro" del motor
  END CATCH
END
```

🎙️ Sobre el CATCH, el profe dijo que, si el error lo provoca un trigger que hace ROLLBACK, se entra al CATCH con la transacción ya deshecha. Pero no todos los errores la deshacen solos (una restricción violada puede dejarla abierta). Por eso la plantilla pregunta `IF @@TRANCOUNT > 0` antes del ROLLBACK: así funciona en los dos casos.

🎙️ El ejemplo integrador de la clase fue una **transferencia bancaria**:

1. Validar los parámetros.
2. Registrar el intento y confirmarlo (una transacción).
3. En otra transacción: sumar al destino (si `@@ROWCOUNT = 0`, la cuenta no existe: ROLLBACK, código 2, RETURN), restar al origen (si no existe: ROLLBACK, código 3, RETURN; acá el ROLLBACK también deshace la suma anterior), registrar los movimientos y marcar la transferencia como confirmada.
4. COMMIT.
5. En el CATCH, si falló por saldo insuficiente (lo controla un trigger), devolver 4; si no, el error del motor.

10.5 Concurrencia: niveles de aislamiento

Cuando dos sesiones trabajan sobre los mismos datos a la vez, pueden aparecer tres problemas:

- **Lectura sucia:** leo un dato que otra transacción modificó pero todavía no confirmó (y que quizás deshaga).
- **Lectura no repetible:** leo una fila dos veces dentro de mi transacción y entre medio otra la cambió.
- **Fantasma:** repito una consulta y aparecen filas nuevas que otra transacción insertó.

El nivel de aislamiento elige cuáles se permiten. Más aislamiento significa menos problemas, pero **más bloqueos**, porque una sesión espera a la otra.

Nivel	Lectura sucia	No repetible	Fantasma
`READ UNCOMMITTED`	puede pasar	puede pasar	puede pasar
`READ COMMITTED` (default)	no	puede pasar	puede pasar
`REPEATABLE READ`	no	no	puede pasar
`SERIALIZABLE`	no	no	no

Se fija con `SET TRANSACTION ISOLATION LEVEL ...`, y queda fijo para toda la sesión. En la práctica se probó con dos ventanas a la vez:

- Una sesión modifica un cliente sin confirmar, y la otra, en `READ UNCOMMITTED`, ve el dato sucio.
- En `REPEATABLE READ`, una sesión que leyó una fila hace esperar a otra que quiere modificarla, hasta el COMMIT.
- Si las dos leen la misma fila y después las dos intentan modificarla, cada una espera a la otra. Es un **deadlock**: SQL Server elige una víctima y cancela su transacción (error 1205).

> **Para llevarte**
>
> - SP = puede modificar. Devolvé un código por OUTPUT.
> - ROLLBACK no corta el SP ni deshace variables: después de cada ROLLBACK, RETURN.
> - UPDATE sin filas no es error: `@@ROWCOUNT`.
> - Orden: defensivo → TRY { BEGIN TRAN ... COMMIT } → CATCH.

Capítulo 11 — Triggers

11.1 Un portero que se despierta solo

🎙️ El profe lo presentó así: un trigger es *"un objeto de base de datos"* que, igual que las funciones y los procedimientos, tiene código procedural, pero que *"se va a estar ejecutando automáticamente cuando cierto evento ocurre"*. Nadie lo invoca. Por eso es el único objeto que garantiza que una regla se cumpla *"sin importar quién modifique"*: otra aplicación, un script o un usuario con acceso directo.

Para describir cualquier trigger, en cualquier motor, hacen falta tres cosas:

	Qué es	Opciones
Evento	qué lo despierta	`INSERT`, `UPDATE`, `DELETE` (uno o varios)
Momento	cuándo corre, respecto del evento	**antes**, **después** o **en vez de**
Acción	qué hace	sentencias SQL, código procedural, llamar a procedimientos o funciones

Lo que tiene SQL Server. De los tres momentos, SQL Server tiene solo dos: `AFTER` (después) e `INSTEAD OF` (en vez de). No hay "antes". 🎙️ Si necesitás uno, usás un INSTEAD OF que primero hace lo tuyo y después **repite él mismo el evento original**. Otros motores, como Oracle, tienen el "antes" (`BEFORE`), te dejan elegir las columnas en la firma y tienen la opción `FOR EACH ROW`, que ejecuta el trigger **una vez por fila**. En SQL Server, en cambio, **el trigger corre una sola vez por instrucción**: si un UPDATE modifica 10 filas, primero se modifican las 10 y después el trigger corre una vez, con las 10 filas a la vista (11.2, regla 1).

Triggers sobre vistas. Los eventos son los mismos, pero el único momento posible es **INSTEAD OF**. Sirve para lo que una vista no puede hacer sola: recibir un INSERT cuando muestra datos de **dos tablas** (11.3, molde F).

Otros triggers. También existen triggers que no dependen de una tabla: al iniciar sesión, ante un error, al levantar o bajar la base. El profe solo los mencionó; no entran en la práctica.

Para qué se usan. 🎙️ La lista del profe:

- **Validaciones** y reglas de integridad que una constraint no puede expresar.
- **Operaciones en cascada**: por ejemplo, borrar una cabecera y que se borren solos sus renglones.
- **Auditoría**: ante un cambio en un campo sensible, guardar el valor viejo, el nuevo, el usuario y la fecha y hora.
- **Replicación**: copiar en caliente a otra base la información muy sensible, por si se rompe un disco.
- **Seguridad**: impedir ciertas operaciones, como modificar o agregarle renglones a una factura que ya está cobrada.

Las dos fotos. Cuando se despierta, el trigger tiene dos **fotos** de lo que pasó, en dos tablas especiales:

- `inserted` : cómo quedaron las filas **después** (las filas nuevas, o la versión nueva de las modificadas).
- `deleted` : cómo eran **antes** (las filas borradas, o la versión vieja de las modificadas).

Operación	`inserted`	`deleted`
INSERT	filas nuevas	vacía
DELETE	vacía	filas borradas
UPDATE	versión nueva	versión vieja

🎙️ Las dos están disponibles **sea AFTER o INSTEAD OF**: aunque en un INSTEAD OF la tabla real todavía no cambió, `inserted` ya tiene los valores que se quieren grabar. Son **de solo lectura**: se consultan, no se modifican. Para relacionar una fila de `inserted` con su versión vieja en `deleted`, hacé un JOIN por la clave primaria.

```
CREATE TRIGGER dbo.tr_nombre ON dbo.Tabla
AFTER INSERT, UPDATE, DELETE        -- o INSTEAD OF ...
AS
BEGIN
  SET NOCOUNT ON;
  ...
END
GO
```

11.2 Las reglas

1. ⚠️ **Se dispara una vez por instrucción, no por fila.** Un `INSERT ... SELECT` de 500 filas dispara el trigger **una** vez, con 500 filas en `inserted`. **Nunca** hagas `SELECT @x = col FROM inserted`: por la regla del 9.2 te quedás con una sola fila y no te enterás. 🎙️ El profe lo marcó como *"un error típico que a veces aparece en los parciales"*: declarar una variable `@importe`, cargarla desde `inserted` y validar solo esa, cuando el UPDATE pudo tocar 10 filas. Pensá en conjuntos: `JOIN inserted`, `IN (SELECT ... FROM inserted)`, `EXISTS`.
2. **AFTER o INSTEAD OF.** Con `AFTER`, la modificación **ya ocurrió**: si no te gusta, `ROLLBACK`. Con `INSTEAD OF`, la modificación **no ocurre**: el trigger se ejecuta en su lugar y decide qué hacer.
3. **Rechazar** = `RAISERROR('mensaje', 16, 1); ROLLBACK TRANSACTION;` (o `THROW 50001, 'mensaje', 1;`).
4. `UPDATE(columna)` dentro del trigger dice si esa columna estaba en el SET. Sirve para salir rápido cuando no te importa el cambio. (🎙️ En Oracle las columnas se ponen en la firma del trigger; en SQL Server entrás siempre y preguntás adentro.)
5. **Corto y simple.** 🎙️ Comparado con un procedimiento, el trigger es *"un poco menos amigable"*:
   - **No tiene parámetros**, ni de entrada ni de salida, así que no puede devolver un código de error ni una descripción para quien hizo la operación.
   - Cuando un trigger hace ROLLBACK, el mensaje del motor es genérico: *"La transacción terminó en el desencadenador. Se anuló el lote."* Te dice que falló, no dónde. Si el trigger tuviera 150 líneas, encontrar la falla sería un problema.
   - Por eso el profe recomienda poner en un trigger solo **sentencias simples, que ya sepas que no pueden fallar** (o que fallen solo en casos muy puntuales): validaciones, auditoría, seguridad, cascadas. La lógica larga, con muchos casos de error, va en un

procedimiento con parámetros de salida.

🎙️ **Antes de escribir un trigger, analizá** (*"de cara a programar y también de cara al parcial"*):

- **Las tablas involucradas.** Si un trigger tiene que reemplazar la funcionalidad de una clave foránea, necesitás **más de un objeto**: uno en la tabla referenciada y otro en la que la referencia. Por eso el enunciado suele decir "cree **el o los** objetos".
- **Todos los eventos** que pueden romper la regla: no solo el INSERT; casi siempre también el UPDATE, y a veces el DELETE.
- **La posibilidad de una modificación masiva** (regla 1).

11.3 Los moldes

Casi todos los ejercicios de triggers de la guía son una de estas formas.

(A) Validar una regla. Ejemplo (T23): una factura no puede tener más de 2 productos compuestos. En un AFTER, la tabla **ya tiene** las filas nuevas, así que se busca la violación en la tabla real, acotada a las facturas que tocó `inserted`:

```
CREATE TRIGGER dbo.tr_max_2_compuestos ON dbo.Item_Factura
AFTER INSERT, UPDATE
AS
BEGIN
   SET NOCOUNT ON;
   IF EXISTS ( SELECT 1
            FROM Item_Factura i
            WHERE i.item_producto IN ( SELECT comp_producto FROM Composicion )
             AND EXISTS ( SELECT 1 FROM inserted ins              -- solo las facturas tocadas
                      WHERE ins.item_tipo = i.item_tipo AND ins.item_sucursal = i.item_sucursal AND ins.item_numero =
i.item_numero )
            GROUP BY i.item_tipo, i.item_sucursal, i.item_numero
            HAVING COUNT(*) > 2 )
   BEGIN
      RAISERROR('Una factura no puede tener mas de 2 productos compuestos', 16, 1);
      ROLLBACK TRANSACTION;
   END
END
```

🎙️ Dos validaciones más que mostró el profe en clase (él usó otra base, con bancos y cuentas; acá están adaptadas a la tuya y probadas).

Un dato que, una vez cargado, no se puede cambiar. Por ejemplo, la fecha de ingreso de un empleado. Se usa `UPDATE(columna)` para no hacer nada si no la tocaron, y `deleted` para saber si ya tenía valor:

```
CREATE TRIGGER dbo.tr_ingreso_inmutable ON dbo.Empleado
AFTER UPDATE
AS
BEGIN
  SET NOCOUNT ON;
  IF UPDATE(empl_ingreso)                    -- ¿estaba en el SET? si no, nada que mirar
  BEGIN
    IF EXISTS ( SELECT 1
          FROM deleted d
          JOIN inserted i ON i.empl_codigo = d.empl_codigo      -- versión vieja con la nueva, por PK
          WHERE d.empl_ingreso IS NOT NULL                      -- ya tenía fecha...
           AND ( i.empl_ingreso <> d.empl_ingreso OR i.empl_ingreso IS NULL ) )   -- ...y la cambiaron
    BEGIN
      RAISERROR('La fecha de ingreso, una vez cargada, no se puede modificar', 16, 1);
      ROLLBACK TRANSACTION;
    END
  END
END
```

Probado: cambiar el salario pasa, volver a grabar la misma fecha pasa, y cambiar la fecha se rechaza. (La versión del profe era más corta: rechazaba si había alguna fila en `deleted` con fecha no nula. Esa también rechaza volver a grabar la misma fecha.)

Que no se repita un valor, salvo los NULL. Por ejemplo: dos clientes no pueden tener el mismo teléfono, pero puede haber muchos sin teléfono. 🎙️ ¿Por qué no una constraint `UNIQUE`? Porque en SQL Server la UNIQUE **trata a los NULL como un valor más** y admite uno solo. En tu base los 3687 clientes tienen el teléfono en NULL, así que ni siquiera se puede crear. El trigger valida solo los teléfonos que se tocaron (el `IN` ignora los NULL):

```
CREATE TRIGGER dbo.tr_telefono_unico ON dbo.Cliente
AFTER INSERT, UPDATE
AS
BEGIN
  SET NOCOUNT ON;
  IF EXISTS ( SELECT c.clie_telefono
          FROM Cliente c
          WHERE c.clie_telefono IN ( SELECT clie_telefono FROM inserted )   -- solo los teléfonos tocados
          GROUP BY c.clie_telefono
          HAVING COUNT(*) > 1 )                          -- en la tabla real, ¿hay dos?
  BEGIN
    RAISERROR('Ya existe un cliente con ese telefono', 16, 1);
    ROLLBACK TRANSACTION;
  END
END
```

(Fuera del parcial existe una alternativa sin trigger: un **índice único filtrado**, `CREATE UNIQUE INDEX ux_clie_telefono ON Cliente(clie_telefono) WHERE clie_telefono IS NOT NULL`. En tu base funciona.)

Fijate en el patrón de las tres validaciones: `IF EXISTS (consulta que encuentra una violación)` → error y ROLLBACK. 🎙️ Para preguntar si hay filas, el profe prefiere `IF EXISTS` antes que `IF (SELECT COUNT(*) ...) > 0`.

(A') Validar en lugar de hacer: INSTEAD OF. Ejemplo (T10, 🎙️ el que el profe dio para hacer en clase): ante el intento de borrar un artículo, verificar que no tenga stock; si no tiene, borrarlo, y si tiene, emitir un error:

```
CREATE TRIGGER dbo.tr_borrar_producto ON dbo.Producto
INSTEAD OF DELETE
AS
BEGIN
   SET NOCOUNT ON;
   IF EXISTS ( SELECT 1 FROM deleted d
            JOIN STOCK s ON s.stoc_producto = d.prod_codigo
            WHERE s.stoc_cantidad > 0 )
   BEGIN
      RAISERROR('No se puede borrar un producto que tiene stock', 16, 1);
      RETURN;                          -- no borro nada: el DELETE original no ocurrió
   END
   DELETE FROM STOCK    WHERE stoc_producto IN ( SELECT prod_codigo FROM deleted );
   DELETE FROM Producto WHERE prod_codigo   IN ( SELECT prod_codigo FROM deleted );
END
```

Fijate que el trigger tiene que hacer él mismo el DELETE, porque INSTEAD OF reemplaza la operación original. 🎙️ El profe lo explicó igual: si hay stock, error (con RAISERROR, ROLLBACK o los dos); si no, primero se borran las filas de `STOCK` (las que tienen cantidad 0 o NULL) y **después** el producto. Supuesto a escribir en el parcial: si se borran varios productos a la vez y alguno tiene stock, se rechaza todo.

(B) Mantener algo precalculado → "recalcular lo afectado".

1. Juntar las claves afectadas de `inserted` **y** de `deleted` (con `UNION`).
2. Borrar esas filas del resumen.
3. Recalcularlas desde la fuente.

¿Por qué no "sumar lo nuevo"? Porque `MAX`, `MIN` y "el cliente que más compró" **no se pueden actualizar de a poco**: si borro la venta que era el máximo, ¿cuál es el nuevo máximo? Hay que mirar todo de nuevo. Recalcular lo afectado sirve para todo, incluidos UPDATE y DELETE. El ejemplo completo es el parcial del 1-11 (apéndice).

(C) Propagar un efecto a otra tabla. Ejemplo: cada venta descuenta stock. Se resta lo que entró (`inserted`) y se devuelve lo que salió (`deleted`), con UPDATE + JOIN:

```
CREATE TRIGGER dbo.tr_venta_descuenta_stock ON dbo.Item_Factura
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
   SET NOCOUNT ON;
   UPDATE s SET s.stoc_cantidad = s.stoc_cantidad - x.cant        -- lo vendido resta
   FROM STOCK s
   JOIN ( SELECT item_producto, SUM(item_cantidad) AS cant FROM inserted GROUP BY item_producto ) x
     ON x.item_producto = s.stoc_producto
   WHERE s.stoc_deposito = '00';                    -- supuesto: se vende desde el depósito 00

   UPDATE s SET s.stoc_cantidad = s.stoc_cantidad + x.cant        -- lo anulado vuelve
   FROM STOCK s
   JOIN ( SELECT item_producto, SUM(item_cantidad) AS cant FROM deleted GROUP BY item_producto ) x
     ON x.item_producto = s.stoc_producto
   WHERE s.stoc_deposito = '00';
END
```

Un UPDATE de renglones pasa por las dos partes: devuelve la cantidad vieja y resta la nueva. El `GROUP BY` sobre `inserted` cubre el caso de una misma instrucción con dos renglones del mismo producto (regla 1).

🎙️ Una **cascada** es una propagación. El ejemplo de clase: si un banco pasa a inactivo, todas sus sucursales pasan a inactivas. Es un AFTER UPDATE sobre bancos con `UPDATE Sucursales SET estado = 0 WHERE banco IN (SELECT codigo FROM inserted WHERE estado = 0)`. El profe remarcó el `IN`: si en una sola instrucción se desactivan tres bancos, se actualizan las sucursales de los tres.

(D) Borrar en cascada: INSTEAD OF DELETE. 🎙️ *"Un clásico"*: al borrar una cabecera, que se borren solos sus renglones. Con un AFTER **no se puede**: la clave foránea de los renglones rechaza el DELETE de la cabecera **antes** de que el AFTER llegue a correr (11.4). Probado en tu base: `DELETE FROM Factura` de una factura con renglones falla por la constraint `R_13`. Con INSTEAD OF, el DELETE original no ocurre y el trigger lo hace en el orden correcto, **primero los hijos y después la cabecera**:

```
CREATE TRIGGER dbo.tr_borrar_factura ON dbo.Factura
INSTEAD OF DELETE
AS
BEGIN
   SET NOCOUNT ON;
   DELETE i                          -- 1) los renglones de TODAS las facturas borradas
   FROM Item_Factura i
   JOIN deleted d ON d.fact_tipo = i.item_tipo AND d.fact_sucursal = i.item_sucursal
            AND d.fact_numero = i.item_numero;

   DELETE f                          -- 2) recién ahora, las cabeceras
   FROM Factura f
   JOIN deleted d ON d.fact_tipo = f.fact_tipo AND d.fact_sucursal = f.fact_sucursal
            AND d.fact_numero = f.fact_numero;
END
```

El segundo DELETE es sobre la misma tabla y el mismo evento del trigger, y **no lo vuelve a disparar**: dentro de un INSTEAD OF, la instrucción repetida va directo a la tabla (🎙️ *"cuando estoy afuera del trigger me lo invoca, pero cuando estoy adentro no"*). (Es un ejemplo para entender el mecanismo: en un sistema real, una factura no se borra.)

(E) Normalizar antes de grabar: INSTEAD OF INSERT. 🎙️ Otro uso del INSTEAD OF: *"no me importa cómo me lo escribas"*. El trigger ajusta los datos y recién ahí hace el INSERT. Por ejemplo, que el detalle de un rubro se guarde siempre en mayúsculas y sin espacios de más:

```
CREATE TRIGGER dbo.tr_rubro_mayusculas ON dbo.Rubro
INSTEAD OF INSERT
AS
BEGIN
   SET NOCOUNT ON;
   INSERT INTO Rubro (rubr_id, rubr_detalle)
   SELECT rubr_id, UPPER(LTRIM(RTRIM(rubr_detalle)))
   FROM inserted;                    -- todas las filas de un saque, no solo la primera
END
```

Probado: insertar `' bebidas sin alcohol'` y `'Lacteos '` en una sola instrucción guarda `BEBIDAS SIN ALCOHOL` y `LACTEOS`. En el ejemplo del profe, el mismo trigger además ponía el banco como activo y la fecha de creación con la del sistema, **ignorando lo que viniera**. 🎙️ Eso es conceptualmente distinto de un `DEFAULT`: el default dice *"si no me lo das, pongo este"*; el trigger dice *"no te lo pido, siempre pongo este"*.

(F) Hacer modificable una vista de dos tablas: INSTEAD OF sobre la vista. Por el 7.7, una vista acepta INSERT solo si es de una sola tabla. Con esta vista de departamentos y zonas:

```
CREATE VIEW dbo.vw_departamentos AS
SELECT d.depa_codigo, d.depa_detalle, z.zona_codigo, z.zona_detalle
FROM Departamento d JOIN Zona z ON z.zona_codigo = d.depa_zona
```

un `INSERT INTO vw_departamentos` falla con *"View or function 'dbo.vw_departamentos' is not updatable because the modification affects multiple base tables"*: el motor no sabe qué insertar en cada tabla. 🎙️ Un INSTEAD OF se lo dice:

```
CREATE TRIGGER dbo.tr_vw_departamentos_ins ON dbo.vw_departamentos
INSTEAD OF INSERT
AS
BEGIN
   SET NOCOUNT ON;
   INSERT INTO Zona (zona_codigo, zona_detalle)              -- la zona, solo si todavía no existe
   SELECT DISTINCT i.zona_codigo, i.zona_detalle
   FROM inserted i
   WHERE NOT EXISTS ( SELECT 1 FROM Zona z WHERE z.zona_codigo = i.zona_codigo );

   INSERT INTO Departamento (depa_codigo, depa_detalle, depa_zona)  -- el departamento, siempre
   SELECT i.depa_codigo, i.depa_detalle, i.zona_codigo
   FROM inserted i;
END
```

Probado con tres departamentos en una sola instrucción, dos de una zona nueva y uno de una zona existente: crea la zona nueva **una sola vez** (por el `DISTINCT`) y los tres departamentos. En la demo del profe se encadenaban tres triggers. El INSERT en la vista lo atendía su INSTEAD OF. Ese trigger insertaba en bancos, que tenía su propio INSTEAD OF de normalizar (molde E), y después en sucursales, que tenía un AFTER que validaba duplicados. Cuando ese último falló, se deshizo **todo**.

11.4 Qué pasa cuando se ejecuta

La transacción ya está abierta. 🎙️ Un procedimiento puede ejecutarse sin ninguna transacción abierta y abrir y cerrar la suya. Un trigger no: *"si yo entré por el evento, la transacción seguro que ya está abierta"*. Es la transacción de la instrucción que lo disparó, y puede tener otras instrucciones antes o después. Por eso:

- Adentro del trigger **no podés confirmarla** (COMMIT): no es tuya.
- Un **ROLLBACK deshace la transacción completa**, no solo la instrucción que disparó el trigger. A un alumno que preguntó, el profe le confirmó que también se deshace lo hecho en otras tablas. Probado: en una transacción con un UPDATE de salario que pasó y después un UPDATE de fecha que el trigger rechazó, **también se deshizo el del salario**.

Masivo o de a uno. 🎙️ No es lo mismo:

```
INSERT INTO T VALUES ('a', '1'), ('b', '2'), ('c', NULL)   -- UNA instrucción: una transacción, el trigger corre 1 vez
```

que tres `INSERT` separados, que en autocommit son tres transacciones y disparan el trigger 3 veces. El profe lo mostró haciendo fallar la tercera fila (NULL en una columna obligatoria). Probado: con la instrucción única la tabla queda **vacía**, porque es una operación atómica. Con tres instrucciones separadas quedan **las dos primeras**.

El orden de las cosas. 🎙️ Cuando llega un INSERT, UPDATE o DELETE:

Sin INSTEAD OF	Con INSTEAD OF
1. se validan las **constraints** (PK, FK, CHECK, NOT NULL)	1. corre el **INSTEAD OF** (las constraints todavía no se miraron)
2. se modifica la tabla	2. la instrucción que él ejecuta valida las **constraints** y modifica
3. corre el **AFTER**	3. corre el **AFTER**

⚠️ Consecuencia: **si falla una constraint, el AFTER nunca se ejecuta**. Probado: un AFTER INSERT en `Departamento` con un PRINT no imprime nada cuando el INSERT viola la clave foránea. Por eso la cascada del molde D tiene que ser INSTEAD OF.

🎙️ **¿Cuántos triggers puede haber?** Para una misma tabla y un mismo evento, **un solo INSTEAD OF** (*"no me puedo estar reemplazando con dos cosas"*). AFTER puede haber varios.

Recursión. 🎙️ ¿Qué pasa si un AFTER UPDATE sobre una tabla hace a su vez un UPDATE sobre la misma tabla? El profe dijo que *"a priori"* sería infinito, que se puede parametrizar cuántos niveles se permiten, y que de todas formas es un error conceptual: *"no tiene ningún sentido"*. En SQL Server, concretamente:

- La **recursión directa** viene **apagada** (la opción `RECURSIVE_TRIGGERS` de la base está en OFF). En tu base, ese AFTER UPDATE no se vuelve a disparar: probado, corre una sola vez.
- La **recursión indirecta** sí ocurre: un trigger en `Zona` que actualiza `Rubro`, y uno en `Rubro` que actualiza `Zona`. Probado: rebotan hasta el límite de **32 niveles**, falla con *"Maximum stored procedure, function, trigger, or view nesting level exceeded (limit 32)"* y se deshace todo.
- Dentro de un **INSTEAD OF**, repetir el evento sobre la misma tabla **no** lo vuelve a llamar. Eso es justamente lo que usan los moldes A', D y E.

Permisos. 🎙️ Como nadie invoca al trigger, no hace falta ningún permiso para "ejecutarlo". Al usuario le alcanza con el permiso sobre la operación que hace (por ejemplo, UPDATE sobre `Cliente`). Lo que hace el trigger adentro no le exige permisos: si el trigger graba una auditoría de cambios del límite de crédito, el usuario **no necesita permiso sobre la tabla de auditoría, ni siquiera de lectura**. *"No sabe que lo están auditando."* Es lo mismo que pasa con los procedimientos y las funciones: el usuario necesita permiso sobre el objeto que invoca, no sobre lo que ese objeto toca.

Para llevarte

- Trigger = evento + momento + acción. Se dispara solo, una vez por instrucción.
- SQL Server: AFTER e INSTEAD OF (no hay "antes"). Sobre vistas, solo INSTEAD OF.
- `inserted` = después, `deleted` = antes. En un UPDATE están las dos. Son de solo lectura.
- Nunca `SELECT @x = ... FROM inserted`. Antes de programar: tablas, eventos y posible modificación masiva.
- Moldes: validar (AFTER + ROLLBACK, o INSTEAD OF), recalcular lo afectado, propagar, cascada con INSTEAD OF DELETE, normalizar con INSTEAD OF INSERT y vista de dos tablas con INSTEAD OF.
- Corre dentro de la transacción de quien lo disparó: ROLLBACK deshace **todo**, COMMIT no se puede.
- Orden: constraints → modificación → AFTER. Si falla una constraint, el AFTER no corre. Un solo INSTEAD OF por tabla y evento.
- Triggers cortos y simples. La lógica larga va en un procedimiento.

Capítulo 12 — Elegir el objeto y diseñar la solución

En el parcial de T-SQL, el enunciado no te dice "hacé un trigger": describe una necesidad, y elegir el objeto es parte de lo que se evalúa.

12.1 El verbo del enunciado

Si el enunciado dice...	Objeto	Por qué
"dado X, retornar/devolver Y", "que pueda usarse en un SELECT / WHERE"	**Función**	devuelve un valor y se usa dentro de consultas; no modifica datos
"devolver un conjunto / listado" para usar como tabla	**Función de tabla**	se consume en el FROM con alias
"directa o indirectamente", "a cualquier nivel", "no se conocen los niveles"	**Función recursiva**	la jerarquía se recorre llamándose a sí misma
"corregir", "reasignar", "completar la tabla", "recategorizar" (una vez)	**Procedimiento**	hace cambios y se ejecuta a demanda con `EXEC`
"que se cumpla automáticamente", "ante una venta...", "no se conoce cómo acceden / n aplicaciones", "no debe grabarse... emitir error"	**Trigger**	es lo único que se dispara solo, sin importar quién modifica
"mantener actualizado / precalculado / siempre disponible"	**Tabla + carga inicial (proc) + triggers + índice**	ver 12.3
"consultar rápido", "optimizar filtros por X"	**Índice** sobre lo que se filtra	
"consulta guardada", "simplificar", "ocultar columnas / dar acceso restringido"	**Vista**	no guarda datos, solo el SELECT (7.7)

12.2 Índices y vistas indexadas

Un **índice** es una estructura que permite encontrar filas sin leer toda la tabla, como el índice de un libro. El **clustered** (uno por tabla, en general el de la clave primaria) además ordena físicamente las filas. Por eso, si la clave primaria de una tabla de resumen es `(anio, mes, producto)`, un `WHERE anio = 2012 AND mes = 7` lee un bloque contiguo. Regla para el parcial: **el índice empieza por las columnas por las que se filtra**.

Una **vista indexada** (`WITH SCHEMABINDING` + `CREATE UNIQUE CLUSTERED INDEX`) sí guarda los datos y se mantiene sola. Para "precalculado" parece ideal, pero tiene **muchas prohibiciones**: no admite `MIN`, `MAX`, `AVG`, `TOP`, subconsultas, `DISTINCT` ni `OUTER JOIN`, entre otras (y si agrupa, exige `COUNT_BIG(*)`). El parcial del 1-11 pedía precio máximo, mínimo y "el cliente que más compró": **imposible con una vista indexada**. Decirlo explícitamente en el parcial suma, porque muestra que lo pensaste.

12.3 Checklist "mantener precalculado"

1. **Tabla** con la granularidad pedida ("por mes, año y producto" → PK `(anio, mes, producto)`).
2. **Índice**: PK **clustered** empezando por las columnas del filtro ("filtradas por mes y año" → `(anio, mes, ...)`).
3. **Carga inicial**: un procedimiento (o script) que llena la tabla con lo que ya existe.
4. **Triggers en todas las tablas fuente** que puedan cambiar el resultado. Preguntate, por cada columna de origen, quién la puede cambiar:
   - `item_cantidad`, `item_precio`, `item_producto` → trigger en `Item_Factura` (INSERT/UPDATE/DELETE).
   - `fact_fecha` (cambia de mes), `fact_cliente` (cambia el top) → trigger en `Factura` (UPDATE).
5. **Supuestos** escritos: desempates, qué pasa si un grupo se queda sin ventas (la fila desaparece).

> **Para llevarte**
>
> - Elegí el objeto por el verbo, y justificá por qué no una vista.
> - El índice empieza por lo que se filtra.
> - Precalculado = tabla + carga inicial + triggers en todas las fuentes + índice.

Capítulo 13 — Plan de práctica

La idea es practicar **por patrón**, no por número. Para cada ejercicio: primero escribí los 5 pasos en comentarios (sujeto, columnas mesa/mandadero, filtros, orden, cardinalidad) y recién después el SQL. Los números son los de tu guía (EjerciciosGuia.ipynb).

Bloque 0 — Cerrar self-joins (30 min)

1. **Ej. 9** (jerarquía, ya lo tenés): rehacelo con el workflow de 5 pasos, sin mirar.
2. **Ej. 15** (pares): fijate bien la diferencia `<>` vs `>`.
3. **Ej. 32** (pares de *familias*): mismo patrón que el 15, un nivel más arriba. Si te sale, self-joins resueltos.

Bloque 1 — El método planilla/mandadero (1 h 30)

4. **Ej. 7** — calentamiento (MAX, MIN, % de diferencia, filtro de stock).
5. **Ej. 12** — **cardinalidad** pura (ventas + stock en el mismo producto). Es el que más enseña.
6. **Ej. 14** — "todos los clientes" + "sin NULLs". Hacelo de las dos formas: LEFT JOIN y UNION ALL (la segunda está resuelta en el 6 b).
7. **Ej. 17** — "mismo mes del año anterior" → año ancla con mes.
8. **Ej. 21** — comparar total de factura contra suma de renglones (mandadero por factura).

Bloque 2 — "El que más..." (1 h)

9. **Ej. 10 / extra B** — está resuelto con UNION ALL en la receta 11: intentalo antes de mirar.
10. **Ej. 23** — por año, el compuesto más vendido + cliente que más lo compró + % del año. Junta las recetas 1 y 7.
11. **Ej. 26** o **Ej. 25** — mismo esqueleto, otro sujeto. Elegí uno.

Bloque 3 — Los parciales (1 h)

12. **Práctica de parcial ej. 1** sin mirar la solución. Después compará con el apéndice.
13. **Parcial 20-09** sin mirar. Ojo con el orden de los filtros.

Bloque 4 — T-SQL y triggers (1 h 30)

0. 🎙️ **Calentamiento de funciones** (son los que el profe asignó y resolvió en clase; T1, T11 y T15 están resueltos en el capítulo 9: intentalos sin mirar y después compará): **T1** (estado del depósito: calculá el % y después un `IF` para armar el texto), **T11** y **T15** (recursivas, 9.6). El **T2** (stock a una fecha) depende de supuestos: él supuso reposición al stock máximo cuando el stock baja del mínimo y recorrió las ventas hacia atrás con un cursor. Si te toca algo así, **escribí los supuestos**. 0b. 🎙️ **Procedimientos** (en clase asignó del T3 al T8): **T3** (dejar un único gerente general y devolver la cantidad de empleados sin jefe que había al comienzo) para practicar (está resuelto en el 10.1; intentalo antes). Después **T4**, y **T5** o **T7** (crear y llenar una tabla desde un SP), usando la plantilla del 10.4.

1. **T10** — INSTEAD OF DELETE (borrar artículo solo si no tiene stock). El trigger más corto: para entender `deleted` (resuelto en el 11.3; intentalo antes).
2. **T23** — molde (A) validar (resuelto en el 11.3; intentalo antes).
3. **T30** — molde (A) con agregación por mes.
4. **T20** — molde (B) precalculado (comisiones por mes). **Es el hermano del parcial del 1-11.**
5. **Parcial 1-11** sin mirar.
6. Si sobra tiempo: **T13** (trigger que usa una recursiva de salarios) y **T16** (molde C, descontar stock).

> Si tenés poco tiempo: hacé 5, 9, 12, 13, 17 y 18. Cubren todos los patrones.

Apéndice — Los tres parciales, pensados en voz alta

> ⚠️ Leelo **después** de intentar cada parcial por tu cuenta (bloques 3 y 4 del plan). Cada uno se resuelve aplicando el método del capítulo 8 o el checklist del 12.3, y el código completo está en SolucionesParcial.sql.

A.1 — Práctica de parcial, ejercicio 1

> Clientes que durante 2 años consecutivos compraron al menos 5 productos distintos. Mostrar código, monto 2012, unidades 2012. Ordenar primero los que compraron solo productos compuestos en algún momento.

Paso 1, sujeto: "devolver clientes" → `FROM Cliente c`. Una fila por cliente.

Paso 2, columnas:

- código → `c.clie_codigo`.
- monto 2012 → otro filtro (año) sobre otra tabla → **mandadero** `ISNULL((SELECT SUM(fact_total) ... 2012), 0)`.
- unidades 2012 → otra tabla (renglones) → **mandadero** con `SUM(item_cantidad)`.

Paso 3, filtro: "2 años consecutivos" (genéricos) → **año ancla** (receta 4) adentro de un `EXISTS`: agrupo las compras del cliente por año, `HAVING COUNT(DISTINCT producto) >= 5` en Y **y** mandadero a Y+1 `>= 5`.

Paso 4, orden: "primero los que compraron solo compuestos en algún momento". Interpretación: existe una factura del cliente cuyos renglones son todos compuestos → `CASE WHEN EXISTS (factura) AND NOT EXISTS (renglón no compuesto) THEN 0 ELSE 1 END` (recetas 5 y 8).

Paso 5, cardinalidad: el FROM es solo `Cliente` y todo lo demás son mandaderos → no hay multiplicación posible.

Código completo: sección 1 de SolucionesParcial.sql (en tu base devuelve 34 clientes).

A.2 — Parcial 20-09-2025 (sin subconsultas en el FROM ni CTE)

> 10 primeros productos con 3 años consecutivos de +20% anual; de esos, solo con stock en ≥75% de depósitos. Mostrar nombre, fecha de última compra, cantidad de clientes que lo compraron en 2025.

Paso 1, sujeto: producto → `FROM Producto p`.

Paso 2, columnas:

- nombre → `p.prod_detalle`.
- última compra → mandadero `SELECT MAX(fact_fecha) ...` (receta: dato de otra tabla).
- clientes 2025 → mandadero `COUNT(DISTINCT fact_cliente) ... AND YEAR = 2025`.

Paso 3, filtros — acá está el examen: dos filtros **en secuencia** ("de los productos seleccionados, mostrar únicamente...") → receta 2:

1. `p.prod_codigo IN (SELECT TOP 10 ...)` y adentro va la racha de 3 años con **año ancla**: agrupo por `(producto, año)`, y en el `HAVING`: `cant(Y+1) >= 1.2 * cant(Y)` **y** `cant(Y+2) >= 1.2 * cant(Y+1)`.
2. Afuera, la condición de stock del 75% (receta 3).

Paso 4: "10 primeros" no dice por qué → asumí uno y **escribilo** (yo usé código de producto; otra opción razonable es por cantidad vendida).

Supuestos para aclarar en el parcial:

- "3 años consecutivos con incremento en cada año": yo lo tomé como años Y, Y+1, Y+2 (dos subas). Si el profe lo quiere como tres subas (Y+1, Y+2, Y+3 contra el anterior), se agrega otro `AND` con un mandadero a `+3`. Es una línea más.
- "Fecha de última compra del producto": la fecha de la última factura que lo incluye.

> En tu base da **0 filas** porque ningún producto tiene stock en más de 12 de los 33 depósitos (36%). Cambiando `0.75` por `0.30` devuelve 2 filas: así comprobás que la lógica anda.

Código: sección 2 de SolucionesParcial.sql. También está la variante con CASE para años fijos.

A.3 — 2do parcial 1-11-2025 (precalculado)

> Mantener precalculado, por mes, año y producto: cantidad total, precio máximo, precio mínimo y cliente que más compró. Siempre actualizado y con acceso optimizado por mes y año.

Paso 1, ¿qué objeto? "Mantener precalculado / siempre actualizado" → tabla + carga inicial + triggers + índice (12.1). ¿Vista indexada? No: MAX, MIN y TOP están prohibidos (12.2). ¿Vista común? No guarda nada: se recalcula en cada consulta, que es justo lo que no quieren.

Paso 2, tabla: granularidad "mes, año, producto" → PK `(anio, mes, producto)`, **CLUSTERED** porque el acceso es "filtrado por mes y año": con la PK en ese orden, `WHERE anio = 2012 AND mes = 7` lee un bloque contiguo del disco.

Paso 3, carga inicial: procedimiento con el `INSERT ... SELECT ... GROUP BY YEAR, MONTH, producto`, y el cliente top con un mandadero TOP 1 (receta 1).

Paso 4, triggers: ¿qué cambios en la fuente alteran el resumen?

- Renglones nuevos, borrados o modificados → trigger en `Item_Factura` AFTER INSERT, UPDATE, DELETE.

- Factura que cambia de fecha (cambia de mes) o de cliente → trigger en `Factura` AFTER UPDATE.
- Ambos con el molde (B) "recalcular lo afectado".

Probado en tu base (dentro de una transacción que después se deshizo): al insertar una factura de julio 2012 con 100000 unidades del producto `00001705` para el cliente `00000`, la fila de (2012, 7, 00001705) pasó de 1724 a 101724 unidades, mínimo 0.01 y cliente top `00000`; al mover la factura a agosto, julio volvió a su valor y apareció la fila de agosto; al borrarla, todo volvió a como estaba.

Código: sección 3 de SolucionesParcial.sql.